



INFORMATIKA
FAKULTATEA
FACULTAD DE
INFORMÁTICA

Trabajo de Fin de Grado

Grado en Inteligencia Artificial

**Detección de tráfico maligno mediante técnicas de
aprendizaje automático en sistemas IDS de red**

Iñaki Moreno Estebanez

Dirección

Iñigo Perona Balda

8 de septiembre de 2026

Resumen

Los sistemas de detección de intrusiones basados en aprendizaje automático alcanzan resultados casi perfectos sobre los conjuntos de datos públicos de referencia. Este trabajo parte de una pregunta incómoda sobre esas cifras: qué miden realmente. Para responderla se construye un *pipeline* completo que convierte capturas de red en bruto en ejemplos etiquetados, en total 17.203.272 conexiones TCP repartidas en seis días del conjunto CSE-CIC-IDS2018, y se comparan tres representaciones del tráfico: la distribución de bytes de la carga útil, las características de flujo y una vista conductual basada en la ráfaga de conexiones por origen. Se procesan además tres días de CIC-IDS2017 con los ataques análogos, que sirven para la validación entre conjuntos de datos.

La primera aportación es metodológica. La *ground-truth* no se hereda de la documentación del conjunto de datos, sino que se reconstruye sobre las propias capturas, y al hacerlo se detecta que la documentación oficial es incorrecta en tres de los siete días analizados. Se documenta además un error que este trabajo cometió dos veces: el tráfico benigno de referencia estaba contaminado por conexiones del propio atacante, lo que producía resultados plausibles y sin sentido.

El resultado central es negativo y reordena el trabajo. Medidas las tres representaciones con la misma métrica y el mismo protocolo, ninguna baja de 0.9186 y el análisis de contenido no baja de 0.9734, de modo que la evaluación dentro de un mismo día está saturada y no discrimina entre ellas. Se propone una explicación, que un único host atacante por día permita memorizar su identidad, y se contrasta y refuta mediante un experimento de exclusión de atacantes con control positivo. Lo que las representaciones aprenden no es el host, sino la herramienta con que se generó el ataque.

La discriminación aparece al perturbar las condiciones. Camuflar los primeros bytes de cada flujo hunde el análisis de contenido de un *recall* de 1.0000 a 0.0025, salvo en el régimen en claro, donde la firma es contenido genuino. Entre días distintos, lo que falla no es la representación sino el umbral: el AUC se mantiene por encima de 0.9724 y bastan 25 flujos etiquetados del dominio nuevo para recuperar el rendimiento sin reentrenar. Y se muestra que la validación entre días no detecta la fuga de identificadores de red, sino que la premia: solo el cambio a otro conjunto de datos la rompe, y al hacerlo revela que dos de los tres regímenes transfieren por la huella de la herramienta y no por el ataque.

Finalmente, se contrastan dos arquitecturas del estado del arte, un *transformer* sobre bytes y una red de atención cruzada, a presupuesto de parámetros comparable. Ninguna mejora los resultados, y cuestan entre 2 y 49 veces más. La conclusión del trabajo es que, sobre estos datos, el cuello de botella no está en la capacidad del modelo sino en la calidad de la etiqueta y en el protocolo con que se mide.

Objetivos de Desarrollo Sostenible

Este trabajo contribuye principalmente al ODS 9, Industria, innovación e infraestructura, y de forma secundaria al ODS 16, Paz, justicia e instituciones sólidas.

ODS 9. Infraestructura resiliente. La meta 9.1 pide desarrollar infraestructuras fiables y resilientes, y la 9.c, aumentar el acceso a las tecnologías de la información. Las redes de comunicaciones son hoy la infraestructura sobre la que se apoyan la sanidad, la energía, el transporte y la administración pública, y su fiabilidad depende de que los ataques se detecten a tiempo. Un sistema de detección de intrusiones es, en ese sentido, infraestructura que protege infraestructura.

La contribución concreta de esta memoria no es un detector mejor, sino algo que condiciona a todos: mostrar que la forma habitual de evaluarlos no mide lo que se cree. Se demuestra que la evaluación dentro de un mismo día está saturada y no distingue entre representaciones, y que buena parte del éxito aparente del análisis de contenido es reconocimiento de la herramienta con que se generó el ataque, no del ataque en sí. Desplegar un sistema validado con ese protocolo produce una falsa sensación de seguridad, que es peor que no tener sistema, porque se confía en una defensa que no defiende. Aportar métodos de verificación, como una *ground-truth* comprobada sobre los datos, pruebas de evasión y medición de la transferencia entre dominios, es contribuir a que la infraestructura digital sea resiliente de verdad y no solo sobre el papel.

Hay además una contribución en eficiencia que la meta 9.4 recoge al pedir industrias sostenibles con uso eficiente de los recursos. El trabajo mide que las arquitecturas más costosas contrastadas no mejoran los resultados y llegan a consumir entre 2 y 49 veces más cómputo. Documentar cuándo un modelo grande no aporta nada evita gasto energético inútil, en un campo donde la tendencia por defecto es escalar el modelo.

ODS 16. Instituciones sólidas. La meta 16.4 aborda la reducción de las corrientes financieras y de armas ilícitas, y la 16.6, el desarrollo de instituciones eficaces y transparentes. El cibercrimen es hoy uno de los vectores principales de actividad ilícita transfronteriza, y su coste se estima en billones de euros anuales. Mejorar la capacidad de detección de intrusiones, y sobre todo la capacidad de saber cuándo esa detección es real y cuándo es un artefacto de la medición, apoya directamente la protección de las instituciones públicas y privadas frente a esa actividad.

La dimensión de transparencia también aplica. Este trabajo publica su metodología, sus datos derivados y sus errores, incluidos los que tuvo que corregir sobre la marcha. La investigación en seguridad que solo reporta sus aciertos contribuye a un cuerpo de conocimiento poco fiable; documentar los resultados negativos y las hipótesis refutadas es una forma modesta pero concreta de sostener la meta 16.6.

Índice de contenidos

Índice de figuras	v
Índice de tablas	vi
1 Introducción	1
1.1. Contexto y motivación	1
1.2. El problema: medir la detección es más difícil que hacerla	1
1.3. Objetivos	2
1.3.1. Objetivo general	2
1.3.2. Objetivos específicos	2
1.4. Aportaciones de este trabajo	3
1.5. Código y reproducibilidad	4
1.6. Estructura de la memoria	4
2 Estado del arte	6
2.1. Detección de intrusiones: de las firmas al aprendizaje	6
2.2. Taxonomía de enfoques	7
2.2.1. Ensembles sobre características de flujo	7
2.2.2. Modelos tabulares profundos	8
2.2.3. Redes neuronales de grafos	8
2.2.4. Payload como imagen: redes convolucionales	9
2.2.5. Transformers y modelos semánticos	9
2.2.6. Modelos de espacio de estados	9
2.2.7. Arquitecturas híbridas y fusión multimodal	10
2.3. Cómo se evalúa en la literatura	10
2.4. Conjuntos de datos públicos y sus defectos	11
2.5. Posicionamiento de este trabajo	13
3 Del PCAP al dataset: metodología	14
3.1. Visión general del pipeline	14
3.2. Extracción del payload con Zeek	15
3.3. Representación vectorial del flujo	16
3.4. El problema del etiquetado: el CSV no trae direcciones	19
3.5. Ground-truth verificada empíricamente	20
3.6. Honestidad del conjunto benigno	22
4 La evaluación intra-día	24
4.1. Marco de comparación	24

4.2.	Los cuatro regímenes: dónde vive la firma	27
4.2.1.	Régimen cifrado: la firma vive en la repetición	27
4.2.2.	Régimen en claro: la firma vive en los bytes	28
4.2.3.	Régimen volumétrico: la firma vive en el agregado	29
4.2.4.	Régimen de periodicidad: la firma vive en el intervalo	30
4.3.	La medición uniforme: todo satura	30
4.4.	Por qué satura: una hipótesis y su refutación	32
4.5.	Aciertan, pero por el motivo equivocado	35
4.6.	El límite del enfoque: lo que el payload no puede ver	36
4.7.	Síntesis: la pregunta correcta	37
5	Generalización, calibración y robustez	39
5.1.	Modelos empleados	39
5.2.	Interpretabilidad: qué mira el modelo	40
5.3.	Robustez ante evasión	41
5.4.	Cómo combinar las dos ramas	42
5.5.	Detección sin etiquetas de ataque	44
5.6.	Generalización entre días del mismo régimen	45
5.6.1.	De diagnóstico a receta: recalibración	46
5.7.	Una vista para la periodicidad, y qué la hace funcionar	46
5.8.	La fuga de identificadores de red	49
5.9.	Generalización entre conjuntos de datos	49
5.10.	¿Mejoran las arquitecturas del estado del arte?	50
6	Conclusiones	53
6.1.	Cumplimiento de los objetivos	53
6.2.	Conclusiones principales	54
6.3.	Lecciones metodológicas	55
6.4.	Limitaciones	56
6.5.	Trabajo futuro	56
	Apéndices	58
A	Declaración del uso de IA Generativa	58
B	Resultados secundarios	60
B.1.	El <i>recall</i> detrás del macro-F1	60
B.2.	Área bajo la curva con intervalos	60
B.3.	Detección no supervisada en términos operativos	61
B.4.	Búsqueda de hiperparámetros	62
	Bibliografía	63

Índice de figuras

3.1.	El pipeline de extracción	15
3.2.	Escala del procesamiento	16
3.3.	De una conexión TCP a los vectores del modelo	18
3.4.	Servicios presentes en cada día	19
3.5.	Composición de clases por día	22
3.6.	El error de límite de ventana	23
3.7.	De qué estaba hecho el tráfico benigno	23
4.1.	El espejismo del 99.96 %	28
4.2.	Distribución de la ráfaga por origen	30
4.3.	La evaluación intra-día está saturada	31
4.4.	Lo único que se invierte son los metadatos per-flujo	32
4.5.	El beacon de la botnet, medido	36
4.6.	La ceguera del payload ante la Infiltration	38
5.1.	Dónde mira el byte-CNN	41
5.2.	El camuflaje separa la huella de herramienta del contenido real	43
5.3.	El híbrido de dos ramas	43
5.4.	Lo que el AUC revela y el umbral fijo esconde	47
5.5.	Recalibración con pocas etiquetas	47
5.6.	El tamaño del atajo de las direcciones IP	51
5.7.	Arquitecturas propias frente a las del estado del arte	52

Índice de tablas

2.1. Familias de modelos y cuáles se han contrastado	7
3.1. Acuerdo entre el servicio identificado por contenido y el puerto	19
3.2. Ground-truth verificada sobre las capturas	21
4.1. Identificación del tipo de ataque web	29
4.2. Contraste de ráfaga por día	29
4.3. Macro-F1 intra-día en los seis días	31
4.4. Leave-one-attacker-out sobre el DDoS del 20-02	33
5.1. Robustez del payload ante camuflaje de los primeros bytes	42
5.2. Estrategias de fusión de las dos vistas	44
5.3. Detección de anomalías entrenada solo con tráfico benigno	44
5.4. Transferencia entre días: umbral fijo frente a AUC	45
5.5. Recalibración con 25 etiquetas del día de destino	46
5.6. Aportación de cada grupo de características	48
5.7. Ablación de las características de inter-arribo	48
B.1. Recall intra-día por vista	60
B.2. AUC-ROC con intervalo bootstrap	61
B.3. Detección no supervisada al 5 % de falsos positivos	61
B.4. Búsqueda de hiperparámetros	62

Índice de algoritmos

1.	Etiquetado de un flujo a partir de la <i>ground-truth</i> verificada	20
2.	Protocolo de comparación entre vistas	26
3.	Leave-one-attacker-out con control negativo y positivo	34

Introducción

1.1. Contexto y motivación

Un sistema de detección de intrusiones en red analiza el tráfico en busca de actividad maliciosa. Durante décadas lo hizo mediante firmas: reglas que describen patrones de bytes asociados a amenazas ya conocidas [1]. El método es preciso y su coste de ejecución es bajo, pero tiene un límite que el mantenimiento no resuelve, porque solo detecta lo que alguien ha analizado y catalogado previamente.

Ese límite se ha agravado por tres motivos. El tráfico circula hoy mayoritariamente cifrado, así que el contenido que la firma inspeccionaría resulta indistinguible del ruido. El malware es polimórfico y altera su apariencia entre infecciones sin cambiar su función. Y las herramientas de ataque se han vuelto accesibles, lo que multiplica el número de variantes en circulación.

Por eso el campo ha girado hacia el aprendizaje automático, que induce el criterio de detección a partir de los datos en lugar de recibirlo escrito. La ventaja esperada es la capacidad de generalizar a variantes no vistas. El coste son los falsos positivos, la opacidad del modelo y la dependencia de datos etiquetados de calidad. Este trabajo se ocupa sobre todo de esto último, que resultó ser el problema más costoso.

1.2. El problema: medir la detección es más difícil que hacerla

El planteamiento inicial de este trabajo era el habitual en la literatura de detección híbrida y se puede enunciar como una tensión entre dos fuentes de información.

Por un lado están las características de flujo: cuántos paquetes componen la conexión, cuántos bytes transporta y cuánto dura. Están siempre disponibles, funcionan igual con tráfico cifrado y son ciegas al contenido. Por otro está la carga útil, es decir, los bytes que la conexión transporta. Contienen el ataque cuando el ataque está en los datos, y no aportan nada cuando el tráfico va cifrado o cuando el ataque no llega a intercambiar contenido.

La hipótesis de partida era que cada tipo de ataque escondería su firma en una de esas dos fuentes, de manera que ninguna bastaría por sí sola y el enfoque híbrido quedaría justificado por esa vía. Para comprobarlo se seleccionaron seis días del conjunto CSE-CIC-IDS2018 que cubren cuatro regímenes distintos: fuerza bruta cifrada, ataques web en claro, denegación de servicio volumétrica y una botnet con actividad periódica.

Medida con cuidado, esa hipótesis no se sostiene, y el motivo es el hallazgo central de esta memoria. Dentro de un mismo día, cualquiera de las representaciones detecta cualquiera de los ataques con un macro-F1 por encima de 0.97, de modo que la comparación está saturada y no permite distinguirlas. Un resultado alto obtenido así no informa sobre la capacidad de detección.

El problema que este trabajo acaba abordando está, por tanto, un nivel más abajo: cómo se mide si un detector ha aprendido lo que se cree que ha aprendido. La respuesta que propone es que hay que perturbar las condiciones de evaluación, dejando que el atacante camufle sus bytes, cambiando de día o cambiando de conjunto de datos, porque es ahí donde las representaciones dejan de ser equivalentes.

La tesis del trabajo sobrevive, pero apoyada en otra evidencia: ninguna vista basta por sí sola, y quienes lo demuestran son la prueba de evasión, la transferencia entre dominios y los ataques que no dejan contenido que analizar.

1.3. Objetivos

1.3.1. Objetivo general

Evaluar, sobre datos reales de red y con un protocolo experimental explícito, en qué condiciones el análisis de la carga útil aporta información que las características de flujo no capturan, y en cuáles no; y determinar qué representación resiste cuando cambian las condiciones de evaluación.

1.3.2. Objetivos específicos

Se enuncian de forma verificable, para poder cerrarlos uno a uno en la sección [6.1](#).

- O1** Construir un *pipeline* reproducible que convierta capturas de red en bruto en un conjunto de ejemplos etiquetados, con la conexión TCP como unidad y varias representaciones por conexión.
- O2** Establecer una *ground-truth* verificada sobre las capturas, en lugar de heredarla de la documentación del conjunto de datos, y documentar las discrepancias que aparezcan.
- O3** Comparar las representaciones con un único protocolo, es decir, con la misma métrica, el mismo balanceo y el mismo tratamiento del tráfico benigno, sobre varios regímenes de ataque, y establecer si la comparación discrimina entre ellas.
- O4** Determinar por qué acierta cada representación, y no solo cuánto, mediante interpretabilidad y pruebas de evasión activas.
- O5** Medir la generalización entre días del mismo régimen y entre conjuntos de datos distintos, e identificar qué componente del modelo falla cuando falla.

- O6** Contrastar arquitecturas del estado del arte con las propias a presupuesto de parámetros comparable, para determinar si el límite de los resultados está en la capacidad del modelo.

1.4. Aportaciones de este trabajo

Hay que separar lo que se toma de la literatura de lo que este trabajo añade, porque la aportación no es una arquitectura nueva.

Lo que se toma del estado del arte

Las representaciones de la carga útil siguen la tradición de PAYL y Anagram [2, 3]. El modelo convolucional sobre bytes sigue la línea de *Deep Packet* [4]. El *transformer* sobre bytes y la fusión por atención cruzada se implementan siguiendo sus referencias respectivas. El escalado de Platt [5] y el error de calibración esperado [6] son métodos estándar. La crítica a los conjuntos de datos públicos parte de Engelen *et al.* [7]. Todo ello se señala en el punto donde se usa.

Lo que aporta este trabajo

Son cinco resultados, agrupados por lo que cada uno permite afirmar.

El primero es una *ground-truth* verificada sobre las capturas para siete días, que corrige la documentación oficial en tres de ellos: la dirección del atacante SSH, el puerto del ataque lento y la continuidad de la actividad de la botnet (sección 3.5).

El segundo es la constatación de que la evaluación intra-día está saturada y no discrimina entre representaciones (sección 4.3). Se propuso una causa, que un único host atacante por día permitiera memorizar su identidad, y se contrastó y refutó con un experimento de exclusión de atacantes que incluye control positivo (sección 4.4).

El tercero es la medición de que el éxito del análisis de contenido suele ser una huella de la herramienta y no del ataque, demostrada por interpretabilidad y por evasión (secciones 5.2 y 5.3).

El cuarto es el diagnóstico de que, entre dominios, lo que se rompe primero es la calibración y no la representación, junto con una receta que cuesta 25 etiquetas (sección 5.6).

El quinto es la constatación de que la validación entre días no detecta la fuga de identificadores de red, sino que la premia (sección 5.8).

A ello se añade un resultado negativo que también es una aportación: ninguna de las arquitecturas avanzadas contrastadas mejora los resultados sobre estos datos, y dos de ellas cuestan entre 2 y 49 veces más (sección 5.10). El límite no está en la capacidad del modelo.

Cómo se llegó a esos resultados

Queda decir algo sobre el método de trabajo, porque las cinco aportaciones tienen un origen común y no es la planificación inicial. Ninguna estaba prevista. Todas aparecieron al desconfiar de un resultado que ya funcionaba.

El patrón se repite y esta memoria lo documenta en lugar de esconderlo. Un primer modelo alcanzaba el 99.95 % de exactitud, y auditarlo contra el tráfico que un detector vería a diario reveló una tasa de falsos positivos del 96 %. El tráfico benigno de referencia estaba contaminado por el propio atacante, dos veces, y el error no se manifestaba como un fallo sino como un número plausible. La saturación intra-día se propuso explicar mediante la memorización del host atacante, se diseñó el experimento capaz de tumbar esa explicación, se le añadió un control positivo para comprobar que el experimento era sensible, y la explicación cayó. Una regla sobre la generalización entre días se enunció con dos observaciones, resultó falsa al añadir la tercera y volvió a cambiar al medirla con la métrica adecuada. Y varias cifras se citaron durante semanas como macro-F1 cuando eran exactitud.

La sección 6.3 recoge esos episodios uno a uno, con lo que cada uno enseñó. Están en la memoria porque un resultado que nadie ha intentado tumbar no es un resultado, y porque la herramienta de verificación descrita más abajo nació precisamente de ellos.

1.5. Código y reproducibilidad

Todo el código con el que se obtuvieron los resultados de esta memoria está disponible en un repositorio público:

<https://github.com/inakimore/deteccion-de-traffic-maligno-mediante-tecnicas-de-aprendizaje-automatico-en-sistemas-ids-de-red>

Contiene el *pipeline* completo, desde el saneado de la captura hasta el etiquetado, las implementaciones de los modelos, los *scripts* de cada experimento y los informes que producen. El fichero `README.md` detalla cómo ejecutar cada paso.

Merece señalarse una decisión de organización, porque sostiene lo que la memoria afirma. Ninguna cifra de este documento está escrita a mano sin respaldo: todas proceden de un módulo único, `resultados.py`, que recoge lo que producen los experimentos, y una herramienta de verificación recorre los capítulos y las tablas comprobando que cada número aparece allí. Esa herramienta tiene además su propio test negativo, que le inyecta un error deliberado y comprueba que lo detecta, para que un resultado en verde no pueda ser un falso verde. La sección 6.3 explica de qué errores nació.

Lo que el repositorio no contiene son las capturas en bruto, que ocupan unos 150 gigabytes y se descargan del organismo que las distribuye [8].

1.6. Estructura de la memoria

El capítulo 2 sitúa el trabajo en su campo. Repasa las familias de modelos empleadas en detección de intrusiones, indicando en cada caso si esta memoria la ha contrastado, y dedica la segunda mitad a los conjuntos de datos públicos y a sus defectos documentados, que es donde enlaza con los capítulos de resultados.

El capítulo 3 describe cómo se pasa de una captura en bruto a un conjunto de ejemplos etiquetados, y se detiene en la parte que suele darse por supuesta: de dónde sale la etiqueta. Contiene la verificación de la *ground-truth* y el episodio del tráfico benigno contaminado.

El capítulo 4 mide. Compara las tres representaciones sobre los seis días con un único protocolo, muestra que la comparación está saturada, contrasta y refuta la explicación que primero se propuso para esa saturación, y termina en los ataques que el análisis de contenido no puede ver.

El capítulo 5 inventaria primero las técnicas de aprendizaje automático empleadas, tanto las propias como las tomadas del estado del arte para contrastarlas, y responde después a la pregunta que el capítulo anterior deja abierta: si dentro del día no se distinguen, cuál resiste cuando cambian las condiciones. Aplica cuatro perturbaciones de exigencia creciente, que son la interpretabilidad, la evasión, el cambio de día y el cambio de conjunto de datos, y cierra contrastando las arquitecturas del estado del arte.

El capítulo 6 cierra los objetivos uno a uno, recoge las lecciones metodológicas, entre ellas las veces que este trabajo se corrigió a sí mismo, declara las limitaciones y propone las líneas de continuación.

Estado del arte

Este capítulo sitúa el trabajo en su campo con una restricción declarada: de cada familia de modelos se dice qué aporta, qué cuesta y si este trabajo la ha contrastado o no. De las siete familias que se repasan, tres se han contrastado experimentalmente y cuatro no. Enumerarlas todas sin distinguirlo produciría un estado del arte más lucido y menos honesto.

La segunda mitad del capítulo trata de los conjuntos de datos públicos, de cómo se evalúan los modelos sobre ellos y de sus defectos conocidos. Ocupa tanto espacio como la taxonomía de modelos, y esa proporción es deliberada: dos de los hallazgos centrales de esta memoria son mediciones de esos defectos, no mejoras de un modelo.

2.1. Detección de intrusiones: de las firmas al aprendizaje

Un sistema de detección de intrusiones en red analiza el tráfico en busca de actividad maliciosa. Durante décadas lo hizo mediante firmas: reglas que describen patrones conocidos, como una cadena concreta en la carga útil o una secuencia de puertos, que un motor contrasta contra cada paquete.

El enfoque tiene una virtud considerable, que es la precisión, y un límite estructural: solo detecta lo que ya se ha visto y catalogado. Frente a variantes polimórficas, tráfico cifrado o ataques nuevos, una base de firmas queda ciega por construcción, y actualizarla exige que alguien haya analizado antes la amenaza [1].

De ahí el giro hacia el aprendizaje automático, que sustituye la regla escrita a mano por un modelo inducido de los datos y aspira a generalizar a variantes no vistas. El giro trae sus propios problemas, entre ellos los falsos positivos, la opacidad del modelo y la dependencia de datos etiquetados, y esta memoria añade uno más, que es su tesis: la dificultad de medir si el modelo ha aprendido lo que se cree.

Hay además dos formas de plantear la tarea, porque la literatura las mezcla y los resultados no son comparables. La detección basada en firmas o en clasificación supervisada parte de ataques etiquetados y aprende a reconocerlos; la detección de anomalías se entrena únicamente con tráfico benigno y marca lo que se desvía. La primera obtiene resultados mucho mejores sobre los conjuntos públicos, pero por construcción solo reconoce lo que se

parece a lo que vio. La segunda es la que aspira a detectar ataques de día cero, a costa de una tasa de falsos positivos mucho más alta. Este trabajo se centra en la primera, y emplea la segunda solo como contraste.

Ese contraste está medido y vale la pena anticiparlo, porque acota cuánto valen las cifras del resto de la memoria. La suposición de que la mayor parte del tráfico es benigno permite entrenar sin etiquetas de ataque, y la sección 5.5 lo hace con dos métodos clásicos, un Isolation Forest y un autoencoder, ajustados únicamente sobre tráfico benigno y evaluados con el área bajo la curva ROC para no depender de un umbral. El resultado es que retirar las etiquetas de ataque cuesta entre veinte y cincuenta puntos según el régimen. Esa es la distancia entre lo que se mide sobre un conjunto etiquetado y lo que un detector encontraría ante una amenaza nueva.

2.2. Taxonomía de enfoques

Las familias que siguen se ordenan de menor a mayor complejidad estructural. La tabla 2.1 las resume y las subsecciones desarrollan cada una.

Tabla 2.1: Familias de modelos para detección de intrusiones y ejemplos representativos de cada una.

Familia	Ejemplos	Contrastada
Ensembles sobre flujo	Random Forest, XGBoost	Sí
Tabular profundo	TabNet	No
Grafos	E-GraphSAGE	No
Payload como imagen	CNN 2D sobre bytes	No
<i>Transformers</i>	ET-BERT, DeBERTa	Sí
Espacios de estados	Mamba-2	No
Híbridos multimodales	Fusión tardía, atención cruzada	Sí

De las siete familias, este trabajo ha contrastado experimentalmente tres. Las otras cuatro se describen porque delimitan el campo, y en cada caso se indica por qué quedaron fuera. Cada subsección expone qué aporta la familia y a qué coste.

2.2.1. Ensembles sobre características de flujo

Random Forest, XGBoost [9] y LightGBM operan sobre vectores de características agregadas del flujo, como la duración, el número de paquetes o los estadísticos de tamaño, y constituyen el estándar de facto del campo. Su rendimiento sobre los conjuntos de datos públicos es difícil de superar y su coste de inferencia es mínimo, lo que los hace además desplegables en línea.

El motivo de su éxito es que las características de flujo son de dimensión baja, están disponibles siempre y son razonablemente informativas: un ataque de denegación de servicio produce conexiones cortas y numerosas, y una fuerza bruta produce conexiones de tamaño casi idéntico. Los ensembles de árboles capturan bien ese tipo de estructura, con umbrales sobre variables individuales, y toleran sin dificultad la mezcla de escalas.

Su límite es la miopía contextual. Una característica agregada no contiene el contenido, así que un ataque cuya firma esté en los bytes queda fuera de su alcance. Tampoco captura relaciones entre flujos, porque cada conexión se clasifica de forma independiente.

Esta familia es la referencia con la que se contrasta en el capítulo 5, y el resultado se puede adelantar, porque matiza la afirmación de supremacía. Dentro del día los ensembles ganan a un perceptrón multicapa sobre la vista tabular, pero por márgenes que van de milésimas a 4.7 puntos, y esa ventaja no sobrevive al cambio de condiciones: al transferir entre días, los tres modelos colapsan por igual salvo cuando se les da acceso a las direcciones IP, que no es una alternativa aceptable sino la medición de un error: la dirección identifica al atacante de este laboratorio y no describe el ataque, y lo que mejora con ella no es la detección sino la memorización. La sección 5.8 lo cuantifica.

2.2.2. Modelos tabulares profundos

TabNet [10] aplica atención secuencial a datos tabulares, buscando la potencia de una red profunda con la interpretabilidad de una selección explícita de características. En cada paso de decisión el modelo elige qué subconjunto de variables mirar, lo que permite explicar sus predicciones sin recurrir a métodos externos. Aplicado a la detección de intrusiones, TabNet-IDS [11] reporta una exactitud media del 97.03 % en CIC-IDS2017 y del 95.58 % en CSE-CIC-IDS2018.

No se ha probado en este trabajo. Sobre los datos aquí empleados la comparación tendría poco valor informativo, porque el capítulo 4 muestra que la vista tabular ya satura y un modelo tabular más potente no tendría margen que recoger. La interpretabilidad que aporta TabNet sí sería útil, pero el trabajo la obtiene por otra vía, mediante saliencia sobre el modelo de bytes.

2.2.3. Redes neuronales de grafos

Las redes sobre grafos, entre ellas E-GraphSAGE [12], modelan el tráfico como un grafo de comunicaciones entre *hosts*: los nodos son máquinas, las aristas son flujos, y el modelo aprende representaciones que incorporan el vecindario de cada nodo. Detectan así anomalías topológicas, es decir, patrones en quién habla con quién que ningún flujo aislado revela.

Es la familia natural para el movimiento lateral, que por definición no vive en un flujo sino en la estructura de las comunicaciones. Su coste es la gestión de estado: construir y mantener el grafo en línea es sustancialmente más complejo que clasificar flujos independientes, y la evaluación exige decidir cómo se segmenta el grafo en ventanas.

No se ha probado, y la ironía salta a la vista, porque es precisamente el enfoque que capturaría el caso que la sección 4.6 identifica como el límite de este trabajo. El reconocimiento de la Infiltration es un abanico de conexiones sin carga útil desde un único origen hacia 612 destinos, es decir, un patrón puramente topológico. Queda como la línea de trabajo futuro con mejor retorno esperado, y se propone en la sección 6.5 por ese motivo y no por completitud.

2.2.4. Payload como imagen: redes convolucionales

Una línea extendida, que arranca con la clasificación de tráfico de *malware* mediante representación aprendida [13], convierte los bytes de la carga útil en una imagen bidimensional, rasterizándolos en una matriz, y aplica una red convolucional 2D. La motivación es práctica: permite reutilizar arquitecturas maduras de visión por computador, con sus pesos preentrenados y sus técnicas de aumento de datos.

La objeción es de fondo y condicionó el diseño de este trabajo. La rasterización destruye la causalidad temporal. Un flujo de red es una secuencia en la que el byte n precede al $n + 1$; al plegarlo en una matriz, bytes distantes en el tiempo quedan adyacentes en el espacio y la convolución 2D los trata como vecinos. La estructura que se le pide aprender al modelo no es la que tienen los datos.

Por eso el byte-CNN de esta memoria opera sobre la secuencia unidimensional, con convoluciones de varios tamaños en paralelo. La decisión resultó acertada por una razón que solo se supo después: la sección 5.2 muestra que la firma se concentra en los primeros 48 bytes, y una convolución con agrupación máxima localiza esa región, mientras que una rasterización la habría repartido entre filas no contiguas.

2.2.5. Transformers y modelos semánticos

ET-BERT [14] y las variantes de DeBERTa aplican preentrenamiento y autoatención al tráfico, tratando el paquete como una secuencia de *tokens* con semántica propia. El preentrenamiento se hace sobre grandes volúmenes de tráfico sin etiquetar, con objetivos autosupervisados análogos a los del procesamiento de lenguaje natural, y el modelo resultante se ajusta después a la tarea concreta con muchos menos ejemplos etiquetados.

Es la familia con mejores resultados publicados en clasificación de tráfico cifrado y malware ofuscado. Su coste principal es la autoatención, cuadrática en la longitud de la secuencia, lo que obliga a truncar los flujos o a segmentarlos. A ello se añade el coste del preentrenamiento, que no siempre se contabiliza al comparar.

Se ha probado, con una restricción importante, que se declara aquí: se contrasta la arquitectura, es decir, un codificador con autoatención a presupuesto de parámetros comparable, y no el preentrenamiento, que es buena parte de lo que aporta ET-BERT. El resultado, en la sección 5.10, es que el *transformer* nunca gana y cuesta entre 36 y 49 veces más tiempo. Esa conclusión vale para esta comparación y no refuta el enfoque en general.

2.2.6. Modelos de espacio de estados

Las arquitecturas basadas en espacios de estados, como Mamba-2, prometen lo mismo que un *transformer* con coste lineal en la longitud de la secuencia en lugar de cuadrático. Aplicadas al tráfico permiten operar directamente sobre los bytes sin *tokenización* ni preentrenamiento, lo que elimina dos decisiones de diseño que en los modelos semánticos condicionan mucho el resultado [15].

No se ha probado. Es la familia más reciente de las siete, su ecosistema de implementaciones es todavía joven, y queda fuera del alcance de este trabajo.

2.2.7. Arquitecturas híbridas y fusión multimodal

Es la familia en la que se sitúa esta memoria. Combina una rama que procesa el contenido con otra que procesa las características de flujo, y la cuestión de diseño es dónde y cómo fusionarlas. Caben al menos tres respuestas: concatenar las entradas antes de procesarlas, fundir las representaciones intermedias que cada rama produce, o dejar que una rama atienda a la otra mediante atención cruzada [16]. Otras propuestas recientes combinan un autoencoder con un clasificador clásico [17].

Este trabajo emplea fusión tardía, en la que cada rama construye su representación por separado y se concatenan antes de la capa de decisión, y contrasta esa elección con la atención cruzada del estado del arte. El resultado es un empate con 2.3 veces menos parámetros (sección 5.10).

Falta un matiz que la literatura no siempre explicita. La fusión ingenua, concatenando las entradas crudas, es peor que cualquiera de las dos ramas por separado, porque las 256 dimensiones del histograma diluyen los cuatro escalares de flujo y el modelo acaba ignorándolos. La diferencia entre fusionar entradas y fusionar representaciones no es un detalle de implementación.

2.3. Cómo se evalúa en la literatura

Los modelos anteriores se comparan casi siempre con el mismo protocolo, y ese protocolo es el objeto de buena parte de esta memoria. Conviene por tanto describirlo antes de pasar a los conjuntos de datos.

La partición

Lo habitual es tomar un conjunto de datos etiquetado, mezclar todos sus flujos y partirlos aleatoriamente en entrenamiento y prueba, o bien aplicar validación cruzada de k particiones. El reparto es aleatorio a nivel de flujo, sin considerar de qué host procede cada uno ni en qué momento se capturó.

Esa elección tiene una consecuencia que rara vez se discute: flujos de la misma conexión lógica, del mismo host y del mismo minuto acaban repartidos entre entrenamiento y prueba. El modelo no tiene que generalizar a una situación nueva, solo a otros ejemplos de la misma situación. El capítulo 4 mide qué queda de las diferencias entre representaciones bajo ese protocolo, y la respuesta es que no queda ninguna.

Las métricas

Se reportan mayoritariamente exactitud, precisión, *recall* y F1, y con frecuencia solo la primera. Sobre conjuntos donde la clase benigna supera el 95 % de los ejemplos, la exactitud es poco informativa, porque un clasificador que responda siempre «benigno» la maximiza. El problema es conocido, y aun así las cifras de exactitud siguen apareciendo como resultado principal.

Menos frecuente es el uso de métricas independientes del umbral, como el área bajo la curva ROC o la curva de precisión-*recall*. La sección 5.6 muestra un caso en que esa ausencia cambia por completo la conclusión: un modelo que con umbral fijo parece no

transferir en absoluto conserva un AUC superior a 0.97: lo que falla es el umbral y no la representación.

Prácticamente ausente está la calibración, es decir, la pregunta de si las probabilidades que el modelo emite se corresponden con las frecuencias reales. Sobre un único conjunto de datos la calibración importa poco, porque el umbral se ajusta una vez; entre dominios distintos es justamente lo primero que se rompe.

Lo que casi nunca se mide

Tres comprobaciones aparecen rara vez en la literatura revisada y son las que este trabajo emplea como eje del capítulo 5.

La primera es la evasión activa, es decir, comprobar qué queda del detector cuando el atacante modifica deliberadamente la parte de la entrada en la que el modelo se apoya. Sin esa comprobación no se puede distinguir una firma del ataque de una firma de la herramienta con que se ejecutó.

La segunda es la interpretabilidad aplicada al diagnóstico, y no solo a la presentación. Saber qué parte de la entrada usa el modelo permite anticipar qué lo romperá, y en este trabajo fue el paso que motivó todos los demás.

La tercera es la transferencia entre dominios, entendida como entrenar en unas condiciones y evaluar en otras distintas. Cuando aparece, suele hacerse entre particiones del mismo conjunto de datos, lo que como muestra la sección 5.8 no basta para detectar los atajos que el propio conjunto introduce.

2.4. Conjuntos de datos públicos y sus defectos

Esta sección es la que más conecta con el resto de la memoria, porque los defectos que describe no son una advertencia general: son el objeto de dos de los hallazgos de este trabajo.

Los conjuntos disponibles

CIC-IDS2017 y CSE-CIC-IDS2018 [18, 8] son los conjuntos de referencia del campo. Se generan en un laboratorio controlado, con perfiles de tráfico benigno sintetizados a partir del comportamiento de usuarios reales y ataques ejecutados con herramientas conocidas y públicas, como Hulk, GoldenEye, Slowloris, LOIC o Patator. Se distribuyen tanto en capturas de red en bruto como en ficheros de características de flujo ya etiquetadas, extraídas con la herramienta CICFlowMeter.

Esa doble distribución explica buena parte de la literatura. Trabajar sobre los ficheros de características es mucho más barato, porque evita procesar centenares de gigabytes de capturas, y por eso la mayoría de los trabajos publicados los usan. El precio es que se pierde el acceso al contenido y se hereda cualquier defecto que la herramienta de extracción introduzca. Este trabajo parte de las capturas en bruto precisamente para evitar las dos cosas.

UNSW-NB15 [19] sigue una filosofía parecida con otra taxonomía de ataques y un generador de tráfico distinto. Malware-Traffic-Analysis.net [20] aporta capturas de infecciones

reales, sin sintetizar, a cambio de no venir etiquetadas de forma sistemática. Ninguno de los dos se ha usado aquí; la sección 6.5 explica por qué el segundo sería el complemento más valioso.

Los cuatro defectos documentados

Engelen, Rimmer y Joosen [7] auditaron CIC-IDS2017 contra sus capturas originales y documentaron cuatro problemas. Los cuatro se reproducen en CSE-CIC-IDS2018, que comparte metodología de generación.

1. Errores de etiquetado. Flujos marcados con la clase equivocada por desajustes entre las ventanas declaradas y la actividad real.
2. Defectos de la herramienta de extracción de características, que produce valores incorrectos en algunas columnas.
3. Ataques «*Attempted*»: intentos que no llegan a completarse y que aparecen etiquetados como ataque aunque no intercambien datos de aplicación.
4. *Shortcut learning*: atributos correlacionados con la etiqueta por accidente del montaje experimental, señaladamente las direcciones IP, que permiten al modelo acertar sin aprender nada del ataque.

Dos de ellos son el objeto de esta memoria

Los dos últimos no se citan aquí como contexto, sino que se miden en los capítulos siguientes, y la conexión se puede enunciar antes de llegar a ellos.

El *shortcut learning* se cuantifica en la sección 5.8. Añadir las direcciones y el puerto de origen a la vista de metadatos produce un macro-F1 de 1.0000 entre días distintos en cinco de los seis pares posibles, porque 15 de las 18 direcciones atacantes del conjunto empiezan por 18. y las 12 víctimas están en 172.31.69.x. Y se añade un resultado que Engelen no reporta: la validación entre días no detecta el atajo, lo premia. Solo el cambio de conjunto de datos lo rompe.

Las etiquetas «*Attempted*» corresponden a lo que la sección 4.6 mide como ceguera del análisis de contenido. En el día de la Infiltration, el 94.4 % de las conexiones del reconocimiento no lleva un solo byte de aplicación, y el ataque de fuerza bruta contra FTP del 14 de febrero deja cero flujos analizables. No es que el enfoque funcione peor sobre esos ataques: es que no tiene nada que leer.

Un defecto añadido por este trabajo

A los cuatro anteriores, el capítulo 4 añade un quinto que la literatura consultada no recoge: la evaluación dentro de un mismo día está saturada. Medidas tres representaciones distintas con la misma métrica y el mismo protocolo sobre seis días, ninguna baja de 0.9186 y el análisis de contenido no baja de 0.9734. La comparación no discrimina, de modo que un resultado alto obtenido así no informa sobre la capacidad de detección.

Esto tiene una consecuencia sobre cómo leer la literatura, y es que buena parte de las cifras publicadas sobre estos conjuntos de datos se obtienen exactamente con ese protocolo.

No implica que esos trabajos estén equivocados, sino que sus diferencias de centésimas no son necesariamente diferencias reales.

2.5. Posicionamiento de este trabajo

De la literatura se toman las arquitecturas y las herramientas. La representación de la carga útil como distribución de bytes procede de la tradición de PAYL y Anagram [2, 3]; el byte-CNN sigue la línea de *Deep Packet* [4]; el *transformer* y la atención cruzada se implementan siguiendo sus referencias respectivas; el escalado de Platt [5] y el error de calibración esperado [6] son estándar; y la crítica a los conjuntos de datos parte de Engelen *et al.* [7].

Lo que este trabajo aporta no es una arquitectura, sino un conjunto de mediciones sobre las condiciones en que esas arquitecturas se evalúan:

- Una *ground-truth* verificada sobre las capturas para siete días, que corrige la documentación oficial en tres de ellos (sección 3.5).
- La constatación de que la evaluación intra-día está saturada y no discrimina entre representaciones, con la hipótesis de la identidad del host contrastada y refutada (secciones 4.3 y 4.4).
- La medición de que el éxito del análisis de contenido suele ser una huella de la herramienta y no del ataque, demostrada por interpretabilidad y por evasión (secciones 5.2 y 5.3).
- El diagnóstico de que, entre dominios, lo que se rompe primero es la calibración y no la representación, con una receta que cuesta 25 etiquetas (sección 5.6).
- La constatación de que la validación entre días no detecta la fuga de identificadores (sección 5.8).

A ello se añade una conclusión que el capítulo 5 sostiene con datos: sobre estos conjuntos, ninguna de las arquitecturas avanzadas contrastadas mejora los resultados, y dos de ellas cuestan entre 2 y 49 veces más. El cuello de botella no está en la capacidad del modelo, sino en la calidad de la etiqueta y en el protocolo de medición. La sección 1.4 desarrolla este posicionamiento.

Del PCAP al dataset: metodología

Los dos capítulos siguientes miden. Este explica sobre qué. Describe cómo se pasa de una captura de red en bruto a un conjunto de ejemplos etiquetados, y se detiene especialmente en la parte que suele darse por hecha, que es de dónde sale la etiqueta.

Esa insistencia no es de trámite. Dos de los hallazgos centrales del trabajo, que un ataque entero fuera invisible por un problema de extracción y que el tráfico benigno de referencia estuviera contaminado por el propio atacante, no son problemas de modelado sino de esta fase. Ninguno se habría detectado mirando métricas.

3.1. Visión general del pipeline

El proceso tiene cuatro etapas, resumidas en la figura 3.1, y cada una descarta información de forma irreversible, así que cada descarte necesita justificación.

1. Saneado de la captura. Las capturas del CSE-CIC-IDS2018 contienen registros que `libpcap` rechaza. Se reescribe la captura antes de procesarla. La sección 3.2 explica por qué este paso no es opcional.
2. Extracción con Zeek. Un *script* propio recorre la captura y emite, por cada conexión TCP, su carga útil de aplicación agregada junto con los metadatos de la conexión.
3. Vectorización. Cada conexión se convierte en varias representaciones numéricas, las vistas que los capítulos siguientes comparan, y se almacena en un fichero `.npz` autocontenido.
4. Etiquetado. A cada conexión se le asigna la clase correspondiente a partir de una *ground-truth* reconstruida sobre las propias capturas, no tomada de la documentación del conjunto de datos.

Dos decisiones atraviesan todo el proceso y se fijan aquí.



Figura 3.1: Las cuatro etapas del proceso, desde la captura en bruto hasta el conjunto de ejemplos etiquetados.

La unidad de ejemplo es la conexión TCP. No el paquete. Cada ejemplo corresponde a una conexión completa, identificada por el uid que asigna Zeek. Un ataque se manifiesta a lo largo de una conversación, y tratar cada paquete como ejemplo independiente haría que un mismo evento apareciera centenares de veces, repartido además entre entrenamiento y prueba.

El fichero de salida es autocontenido. Contiene, por flujo, el histograma de bytes, la secuencia de bytes, la entropía, la 5-tupla de direcciones y puertos, la marca de tiempo y la etiqueta. Esto permitió más adelante derivar vistas nuevas, la ráfaga por origen y el inter-arribo, sin volver a procesar las capturas. Resultó decisivo, porque los datos intermedios ocupan decenas de gigabytes por día y se borran tras la vectorización.

3.2. Extracción del payload con Zeek

Zeek [21] se emplea como analizador de protocolos. Identifica el servicio por contenido, reensambla el flujo TCP y expone la carga útil de aplicación. Sobre él se ejecuta un *script* propio que, por cada conexión, acumula hasta 64 kilobytes de carga útil y los emite junto con el servicio identificado, la 5-tupla, el número de paquetes, los bytes totales y la duración.

La elección de Zeek frente a alternativas como CICFlowMeter, que es la herramienta con la que se generan los ficheros de características que acompañan al conjunto de datos, responde a dos motivos. El primero es que Zeek da acceso a la carga útil, y no solo a estadísticos agregados, que es el objeto de este trabajo. El segundo es que su identificación de servicio se basa en el contenido y no en el puerto, lo que permite la corrección que describe la sección 3.3.

El saneado no es opcional, y descubrirlo costó un ataque entero

Durante las primeras semanas del trabajo el ataque de fuerza bruta contra SSH no aparecía en los datos extraídos. El día se procesaba sin errores, Zeek terminaba correctamente y el resultado contenía únicamente tráfico benigno.

La causa era una truncatura silenciosa. Zeek se detenía a las 18:05 UTC al encontrar un registro que libpcap rechaza, y descartaba sin avisar todo lo que venía después, que era precisamente la tarde en que ocurría el ataque. No se emitía ningún mensaje de error: el proceso terminaba con éxito sobre una fracción de la captura. Al reescribir la captura antes de procesarla se recuperan los 2.9 millones de paquetes reales y el ataque aparece completo.

La lección se incorporó al procedimiento como una comprobación obligatoria, que consiste en validar que la cobertura temporal de lo extraído coincide con el rango real de la captura. Un fallo así no produce un error, produce un conjunto de datos plausible y equivocado, que es mucho peor.

Procesamiento por lotes bajo restricción de disco

Los días completos del CSE-CIC-IDS2018 se distribuyen en centenares de capturas que suman decenas de gigabytes, y los ficheros intermedios que genera Zeek ocupan aproximadamente 1.48 veces lo que ocupa la captura de origen. Un día completo no cabe en disco si se procesa de una vez.

El procesamiento se organiza por lotes. Se ordenan las capturas de mayor a menor, se procesa cada una y se elimina la captura en cuanto su salida está verificada, comprobando antes que el número de bytes extraídos es el esperado. El proceso se detiene de forma limpia si el espacio libre baja de un umbral, y nunca borra nada si la extracción ha fallado.

Es una restricción de ingeniería, pero condicionó qué días se pudieron procesar y merece constar. La figura 3.2 da la escala del trabajo en número de capturas, gigabytes procesados y flujos resultantes.

6	~150 GB	9.4 M	3
días procesados	de capturas PCAP	flujos TCP vectorizados	regimenes de firma
3 de CSE-CIC-IDS2018 + 3 de CIC-IDS2017	saneadas y procesadas con Zeek en Docker	histograma, entropía y secuencia por conexión	cifrado, en claro y volumétrico

Figura 3.2: Capturas, volumen en disco y flujos extraídos por día.

3.3. Representación vectorial del flujo

De cada conexión se derivan cinco representaciones, que la figura 3.3 resume. Las tres primeras son las que los capítulos siguientes comparan como vistas; las dos últimas se añadieron al identificar regímenes que las anteriores no capturaban.

Vale la pena situarlas antes en la taxonomía clásica de la detección de intrusiones, que distingue tres tipos de características [22]. Las *intrínsecas* se derivan de la cabecera de los paquetes de una misma conexión y la describen por sí sola: duración, número de paquetes y bytes transportados. Las de *contenido* se derivan de la carga útil, es decir, de lo que la conexión transporta por encima de la capa de transporte. Las de *tráfico* no se calculan sobre

una conexión sino sobre un grupo de ellas, agregando por origen o por destino en una ventana temporal.

Las cinco representaciones de este trabajo se reparten sobre esos tres tipos. El histograma de bytes y la secuencia de bytes son de contenido, y a ellas se añaden los diecinueve descriptores interpretables que se describen más abajo. Los metadatos de flujo son intrínsecos. La ráfaga por origen y el inter-arribo son de tráfico, porque ninguna de las dos se puede calcular mirando una sola conexión.

Hay una colisión de nombres que declarar para que no confunda. Lo que esta memoria llama metadatos de flujo son características intrínsecas en la taxonomía anterior, no de tráfico: describen la conexión aislada. El contraste que atraviesa los capítulos siguientes, entre contenido y flujo, es por tanto un contraste entre el segundo tipo y el primero, y las características de tráfico propiamente dichas entran en la vista conductual.

Antes de enumerarlas, una precisión sobre qué se calculan las dos primeras, porque la unidad es la conexión y no el paquete. El *script* de Zeek emite una fila por cada segmento TCP que transporte datos, en el orden en que aparece en la captura y en ambos sentidos. La vectorización recorre esas filas agrupando por uid y concatena las cargas útiles en ese mismo orden cronológico, sin separar la dirección cliente-servidor de la inversa. Sobre esa concatenación se calculan las dos representaciones de contenido: el histograma acumula hasta 64 kilobytes por conexión, que es un tope de memoria, y la secuencia se queda con los primeros 256 bytes, rellenando con ceros las conexiones más cortas.

Los 256 bytes de la secuencia no son, por tanto, el primer paquete, sino el comienzo del diálogo. En la práctica suelen abarcar el primer segmento con datos y parte del siguiente, que es donde vive la cabecera del protocolo de aplicación: el *banner* SSH, la línea de petición HTTP o el saludo TLS. La sección 5.2 muestra que es justamente ahí donde el modelo mira.

Histograma de bytes. Frecuencia de cada uno de los 256 valores posibles en la carga útil agregada, normalizada, más la entropía de esa distribución. Es una representación sin orden: descarta la secuencia y conserva la composición. Sigue la tradición de PAYL [2].

Secuencia de bytes. Los primeros 256 bytes de la carga útil, en orden, como enteros. Conserva la estructura local que el histograma pierde, y es la entrada de los modelos convolucionales y recurrentes.

Metadatos de flujo. Cuatro escalares de la conexión: número de paquetes, bytes totales, longitud de la carga útil y duración. Se les aplica $\log(1 + x)$ porque sus distribuciones tienen colas muy pesadas.

Ráfaga por origen. Número de conexiones que el mismo origen ha abierto en el segundo, los cinco segundos y los treinta segundos previos. Se calcula de forma causal, mirando solo al pasado, porque un detector en línea no puede consultar el futuro.

Inter-arribo. Seis características derivadas de los intervalos entre conexiones consecutivas del mismo origen: el intervalo anterior, el siguiente, la media y el coeficiente de variación en una ventana local, una medida de regularidad y la fracción de intervalos cercanos a la mediana.

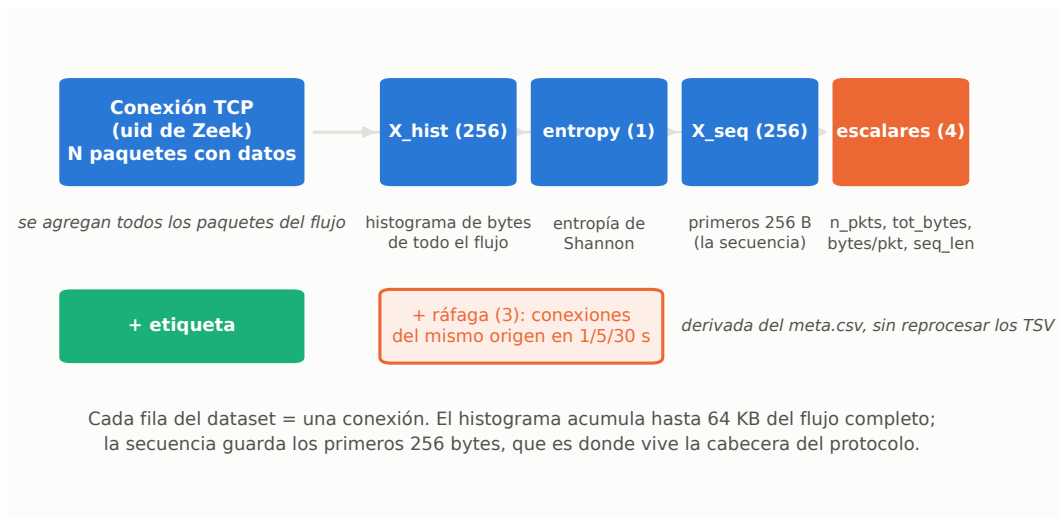


Figura 3.3: Las representaciones que se derivan de una misma conexión.

Descriptores del payload tomados de la literatura

Al histograma y la secuencia se añade un tercer conjunto de características sobre la carga útil, esta vez interpretables y tomadas de la literatura de detección por contenido. Son diecinueve descriptores agrupados en cuatro familias: medidas de aleatoriedad, como la entropía de Shannon normalizada, el índice de coincidencia o el ratio de compresión; medidas de forma de la distribución, como la frecuencia del byte dominante, la masa de los cuatro dominantes o la distancia a la distribución uniforme; medidas sobre 2-gramas, en la línea de Anagram [3]; y medidas léxicas, como la fracción de bytes imprimibles, de alfanuméricos, de caracteres de control o de bytes no ASCII, orientadas a detectar inyecciones y *shellcode*.

La motivación es doble. Por un lado permiten unir el análisis de contenido a las características que produce Zeek en un mismo vector de dimensión manejable. Por otro son interpretables una a una, cosa que los 256 valores del histograma no son. El resultado, que el capítulo 5 recoge, es que los diecinueve descriptores rinden aproximadamente igual que las 257 dimensiones del histograma.

Qué se excluye deliberadamente

Las direcciones IP y los números de puerto no forman parte de ninguna vista. La razón se desarrolla en la sección 5.8: en este conjunto de datos son un atajo casi perfecto y además sobreviven a la validación entre días. Se excluyen desde el principio, salvo en un experimento donde se incluyen deliberadamente y como control declarado, para medir el tamaño del atajo.

La exclusión no es una precaución conservadora, y se enuncia como lo que es: usar la dirección de origen como característica no es correcto. No describe el ataque sino la identidad de quien lo lanzó en este laboratorio concreto, de modo que un modelo que la use no aprende a detectar nada, aprende una lista de culpables. Y esa lista no es transferible: cambia con la red, con el escenario y con el atacante, así que un resultado obtenido con ella no dice nada sobre lo que el detector haría en otro sitio. Lo mismo vale para el puerto, que

3.4. El problema del etiquetado: el CSV no trae direcciones

depende de la configuración de cada red. Cuando en esta memoria aparecen direcciones, lo hacen como identificador para etiquetar y auditar, nunca como entrada del modelo, excepto en el control positivo declarado de la sección 4.4 y en la medición del atajo de la sección 5.8, que existen precisamente para cuantificar el error que se estaría cometiendo.

La selección por servicio, medida

El servicio se toma del análisis de contenido de Zeek y no del número de puerto. Esta corrección se aplicó a mitad del trabajo y obligó a reevaluar todos los resultados anteriores.

La pregunta que quedaba abierta era cuánto cambiaba en la práctica, y se puede medir: basta comparar, flujo a flujo, la etiqueta de servicio que da el contenido con la que daría el puerto. La tabla 3.1 recoge el resultado sobre los días en que ambas son comparables.

Tabla 3.1: Para cada día, número de flujos en que ambas identificaciones son comparables, cuántas coinciden y cuántas se contradicen.

Día	Comparables	Confirman	Acuerdo	Contradicen
14-02 (SSH)	2.200.888	2.200.419	99.98 %	469
22-02 (Web)	2.769.964	2.769.217	99.97 %	747
16-02 (DoS)	4.038.401	4.037.957	99.99 %	444
04-07 2017 (SSH)	98.154	98.033	99.88 %	121
05-07 2017 (DoS)	260.796	259.410	99.47 %	1.386
06-07 2017 (Web)	81.352	81.330	99.97 %	22

El acuerdo supera el 99.4 % en los seis días, así que ningún resultado previo cambia. El valor de la corrección es metodológico: los resultados dejan de depender de una convención de configuración que en otra red podría ser distinta. La figura 3.4 muestra, por día, qué protocolos aparecen identificados por contenido.

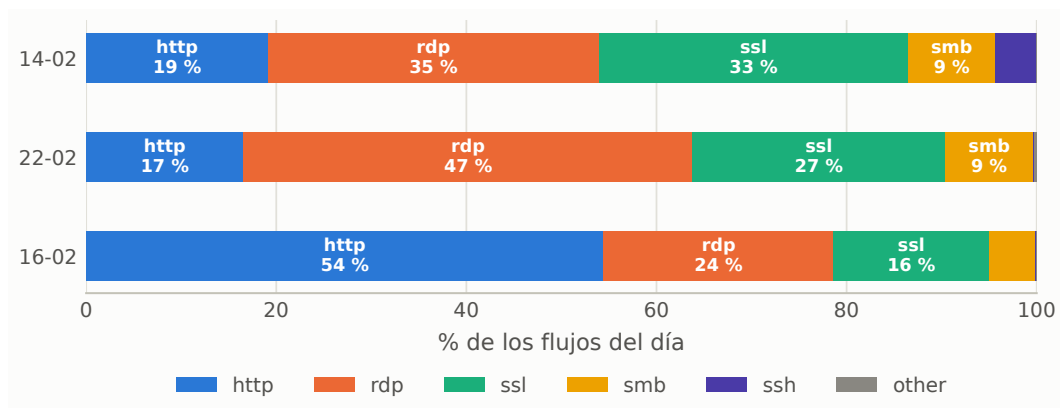


Figura 3.4: Protocolos identificados por contenido en cada uno de los días procesados.

3.4. El problema del etiquetado: el CSV no trae direcciones

El CSE-CIC-IDS2018 se distribuye con ficheros CSV de características de flujo ya etiquetadas. Podría parecer que basta con cruzarlos con las capturas, pero no basta, por una razón simple: esos CSV no incluyen las direcciones IP. Traen marcas de tiempo y características agregadas, pero no permiten identificar qué conexión de la captura corresponde a cada fila.

La etiqueta, por tanto, hay que reconstruirla. El procedimiento adoptado modela cada ataque como un par $\{\text{atacante}, \text{víctima}\}$: puerto acompañado de una ventana temporal, y asigna la clase de ataque a toda conexión que encaje en ambos criterios. Los ataques y sus parámetros se declaran en un módulo único, para que la *ground-truth* sea explícita, versionada y revisable, en lugar de estar dispersa por los *scripts* de evaluación.

El algoritmo 1 recoge el procedimiento. Su aparente sencillez esconde los dos puntos donde este trabajo se equivocó: la comprobación de la ventana en la línea 4, cuyos límites hay que verificar contra la actividad real y no contra la documentación, y el hecho de que un flujo que no encaja en ningún ataque se etiqueta como benigno por defecto, en la línea 9, que es exactamente lo que contamina la clase negativa cuando la ventana se queda corta.

Algoritmo 1 Etiquetado de un flujo a partir de la *ground-truth* verificada

Entrada: Flujo f con origen o , destino d , puerto p y marca de tiempo t ; conjunto de ataques A del día

Salida: Etiqueta de f

```

1: para todo ataque  $a \in A$  hacer
2:    $par \leftarrow \{o, d\}$ 
3:   si  $par \subseteq a.direcciones$  y  $p = a.puerto$  entonces
4:     si  $a.inicio \leq t \leq a.fin$  entonces
5:       devolver  $a.etiqueta$ 
6:     fin si
7:   fin si
8: fin para
9: devolver BENIGNO

```

El modelo tiene un límite que declarar, porque determinó qué días se pudieron usar. Un ataque en el que un único origen contacta con centenares de destinos distintos no se puede expresar como un par de direcciones, y ese es exactamente el caso del reconocimiento de la Infiltration. Por eso ese día se analiza en la sección 4.6 sin construir un conjunto etiquetado.

La zona horaria como fuente de error

Las capturas están en UTC y la documentación del conjunto de datos en hora local, con un desfase que no viene declarado de forma inequívoca. Un error de una hora desplaza la ventana y reetiqueta miles de flujos en ambos sentidos, marcando como ataque tráfico legítimo y como benigno actividad del atacante.

El desfase se verificó empíricamente en lugar de asumirse. Se localizó el primer paquete del atacante en la captura y se comparó con la hora declarada. Para el CSE-CIC-IDS2018 resulta ser UTC−4: una anotación de las 08:31 locales corresponde a las 12:30 UTC en la captura. Para el CIC-IDS2017, empleado en la validación complementaria, resulta ser UTC−3. Son distintos, y darlo por supuesto habría corrompido el etiquetado de tres días.

3.5. Ground-truth verificada empíricamente

Esta sección recoge la aportación metodológica del capítulo. La *ground-truth* de los siete días no se tomó de la documentación oficial, sino que se reconstruyó sobre las capturas y se verificó una por una. La tabla 3.2 recoge el resultado.

Tabla 3.2: *Ground-truth* verificada empíricamente sobre las capturas, no tomada de la documentación del conjunto de datos. Horas en hora local (UTC−4).

Día	Ataque	Atacante	Víctima
14-02	SSH-Bruteforce	13.58.98.64	172.31.69.25:22
15-02	GoldenEye/Slowloris	18.219.211.138 y 18.217.165.70	172.31.69.25:80
16-02	Hulk/SlowHTTPTest	18.219.193.20	172.31.69.25:80
20-02	DDoS-LOIC-HTTP	10 direcciones	172.31.69.25:80
22-02	Web (BF/XSS/SQLi)	18.218.115.60	172.31.69.28:80
02-03	Bot (Ares)	18.219.211.138 (C2)	10 bots en 172.31.69.x
28-02	Infiltration	13.58.225.34 (C2)	172.31.69.24

Cómo se localiza a un atacante sin conocerlo

El procedimiento no parte de la dirección documentada, precisamente para poder contrastarla. Se ordenan los orígenes de la captura por número de conexiones, porque los ataques de fuerza bruta y volumétricos dominan por varios órdenes de magnitud, y se contrasta el candidato con la ventana declarada.

Cuando no hay un origen dominante el procedimiento se invierte. Es el caso del día de la botnet, donde el canal de mando y control mueve poco tráfico y queda enterrado entre los orígenes legítimos: allí se extrajo primero la carga útil y se buscó en ella el patrón del canal, para identificar después la dirección. El día de la Infiltration exigió un tercer procedimiento, sobre el registro de conexiones y no sobre la carga útil, porque el reconocimiento no exporta datos de aplicación.

La documentación oficial falló tres veces

Este es el resultado que justifica el esfuerzo. En tres de los siete días la documentación del conjunto de datos no coincide con lo que contienen las capturas.

En el día 14-02 la dirección del atacante SSH documentada no es la que aparece en la captura, y además la ventana declarada no cubre toda la actividad continua del atacante.

En el día 16-02 el ataque SlowHTTPTest se dirige al puerto 21 y no al 80, que es lo que indica la documentación. Es también el ataque que no deja ni un flujo con carga útil analizable.

En el día 02-03 la documentación parte la actividad de la botnet en dos franjas horarias. Medida sobre la captura, la actividad es continua, con un hueco máximo de 32.6 segundos. Etiquetar según la documentación habría marcado como benignas varias horas de tráfico de la botnet.

No es una crítica al conjunto de datos, que es una contribución valiosa y de uso generalizado. Es una constatación con consecuencias prácticas: una etiqueta heredada sin verificar es una fuente de error tan seria como un modelo mal diseñado, y a diferencia de este último no se manifiesta como un fallo, sino como un resultado creíble. La literatura reciente ha llegado a la misma conclusión al reetiquetar CIC-IDS2017 [7].

El desbalanceo que resulta

La figura 3.5 muestra la composición de clases de cada día una vez aplicada la etiqueta verificada. El desbalanceo es extremo y muy desigual entre días: el día web tiene 203 flujos de ataque entre 2.7 millones, mientras el del DoS-Hulk tiene 1.8 millones de ataque sobre 4 millones de flujos. Es la razón por la que el capítulo 4 balancea 1:1 antes de comparar y no reporta exactitud global.

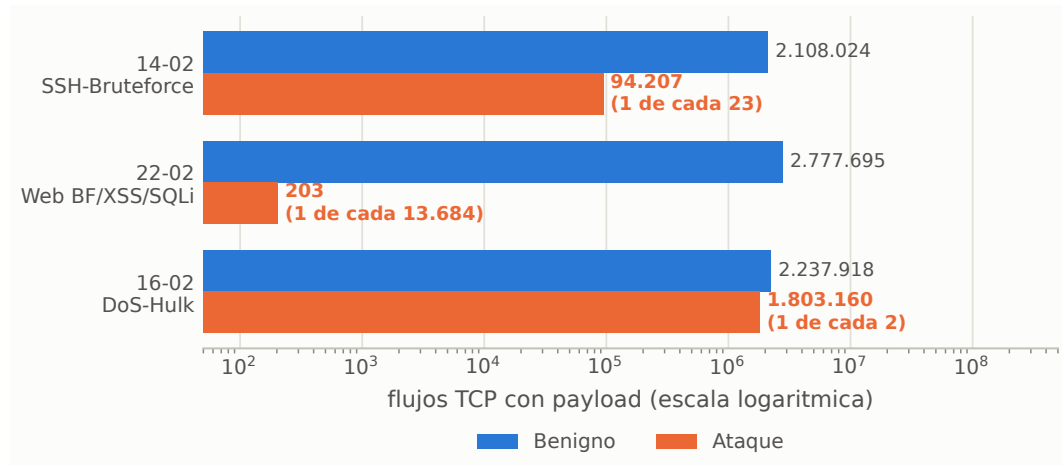


Figura 3.5: Flujos de ataque frente a flujos benignos en cada día, tras aplicar la *ground-truth* verificada.

3.6. Honestidad del conjunto benigno

Queda un error del que este trabajo fue víctima dos veces, y que merece una sección propia porque es el más difícil de detectar de todos los descritos.

Cuando la ventana etiquetada no cubre toda la actividad del atacante, las conexiones que quedan fuera no desaparecen: se quedan en el conjunto de datos etiquetadas como benignas. El resultado es un conjunto de referencia contaminado por el propio ataque, y cualquier comparación hecha sobre él enfrenta el ataque consigo mismo.

La figura 3.6 ilustra el mecanismo, y ocurrió dos veces. En el día del SSH-Bruteforce, 1.589 de los 2.022 flujos etiquetados como benignos del puerto 22 eran del propio atacante, y solo 433 procedían de terceros. En el día del DoS-Hulk, el flujo de ataque es continuo hasta las 17:58:22 UTC mientras la ventana terminaba a las 17:52:59, con lo que quedaban fuera 740.821 flujos del atacante, el 65.1 % de todo el tráfico HTTP etiquetado como benigno de ese día.

Por qué no se detecta mirando las métricas. Porque los resultados que produce son plausibles. Con el benigno contaminado, ninguna vista superaba el 0.78 en el día del SSH, y ese techo se interpretó durante semanas como una propiedad del problema, como si los metadatos no dieran para más, cuando era un artefacto del etiquetado. Un número bajo no levanta sospechas del mismo modo que un 1.0000, y ahí está la trampa.

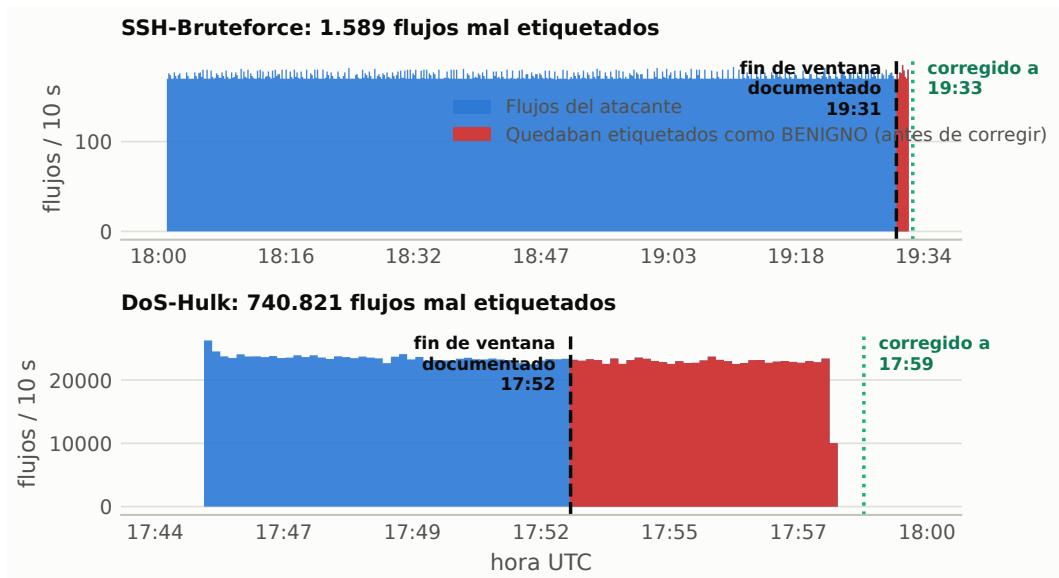


Figura 3.6: Actividad del atacante frente a la ventana etiquetada. Las conexiones que quedan fuera no desaparecen: pasan a engrosar la clase benigna.

Cómo se detecta. Auditando el origen de cada flujo de la clase negativa, y comprobando que la ventana etiquetada cubre la actividad continua del atacante sobre el conjunto completo y no sobre una captura suelta. La figura 3.7 descompone el tráfico etiquetado como benigno según su procedencia real. La corrección consistió en arreglar las ventanas y reetiquetar esos flujos como lo que eran, actividad del ataque.

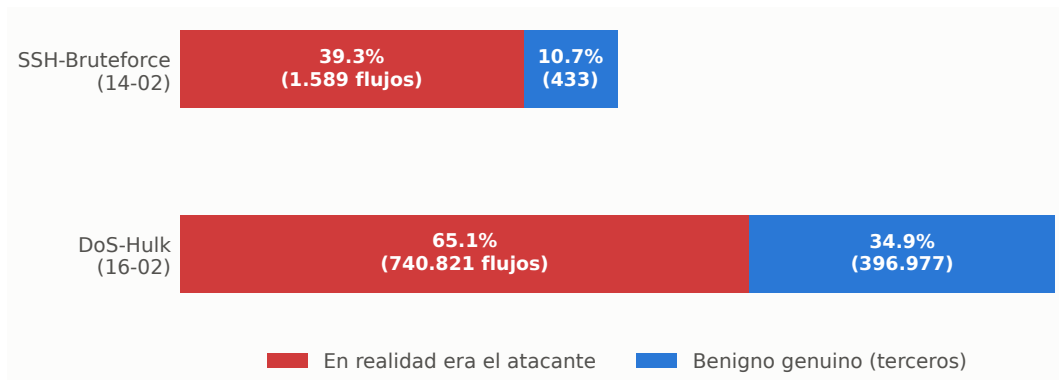


Figura 3.7: Descomposición del tráfico etiquetado como benigno según su procedencia real.

Una consecuencia para la reproducibilidad. Al corregir el etiquetado en el propio conjunto de datos, el escenario contaminado deja de ser reconstruible. Las cifras que en su momento lo describieron son históricas y no comparables con las de los capítulos siguientes, y así se declaran cada vez que se mencionan.

Esta es, junto con la verificación de la *ground-truth*, la razón por la que este capítulo ocupa el lugar que ocupa en la memoria. Los dos capítulos que siguen no tendrían valor sin él, porque medirían con precisión sobre datos equivocados.

La evaluación intra-día: por qué no discrimina

Este capítulo mide. Toma los seis días procesados del CSE-CIC-IDS2018, aplica a todos el mismo protocolo y las mismas tres representaciones de los datos, y compara. El resultado no es el que se esperaba al empezar, y es el hallazgo central del trabajo: dentro de un mismo día el problema está saturado. Cualquiera de las vistas detecta cualquiera de los ataques con un macro-F1 por encima de 0.97, de modo que la comparación no distingue entre ellas.

Eso no es un fracaso de la medición sino un resultado sobre el conjunto de datos, y tiene una causa identificable que la sección 4.4 documenta. Su consecuencia ordena el resto de la memoria: si la evaluación dentro del día no discrimina, la pregunta de qué vista es mejor hay que plantearla en otro sitio. Ese otro sitio, la robustez ante evasión, la transferencia entre dominios y la calibración, es el capítulo 5.

4.1. Marco de comparación

Todas las comparaciones de este capítulo y del siguiente siguen el mismo protocolo, que se fija aquí para no repetirlo. Las decisiones que lo componen no son neutrales: cada una corrige un sesgo concreto que se detectó experimentalmente durante el trabajo, y sin ellas los resultados serían sistemáticamente optimistas.

Selección por servicio, no por puerto

El subconjunto de flujos sobre el que se evalúa cada día se define por el servicio identificado por contenido, con las mismas firmas que emplean los analizadores de Zeek: el *banner* SSH, los métodos HTTP, el registro TLS o los saludos SMTP y FTP. No se define por el número de puerto. El puerto depende de la configuración de cada red y no es, por tanto, un atributo independiente del contexto: un mismo servicio puede escucharse en puertos distintos y un mismo puerto alojar servicios distintos. Usarlo como criterio ata los resultados a la topología del laboratorio.

La corrección no alteró ninguna conclusión previa, porque en un entorno controlado el contenido confirma al puerto casi siempre, pero sí cambia lo que se puede afirmar. Y

cambia algunos recuentos: en el día del SSH-Bruteforce el tráfico benigno genuino son 433 flujos si se selecciona por el puerto 22 y 358 si se selecciona por contenido. Las cifras que se reportan a partir de aquí son siempre las de la selección por contenido.

Un flujo es una conexión TCP

La unidad de ejemplo es la conexión TCP, identificada por el uid que asigna Zeek, y no el paquete. Un ataque se manifiesta a lo largo de una conversación, no en un datagrama aislado, y agregar por conexión evita que un mismo evento aparezca centenares de veces en el conjunto de entrenamiento y una sola vez en el de prueba.

Balanceo 1:1, validación cruzada y repeticiones

El tráfico benigno representa más del 95 % de los flujos en todos los días analizados, así que la exactitud global es una métrica engañosa: un clasificador que responda siempre «benigno» la maximiza. Cada comparación se hace sobre un subconjunto balanceado 1:1 por submuestreo aleatorio de la clase mayoritaria, con un tope de 5.000 ejemplos por clase. Con ese balanceo el azar corresponde exactamente a 0.5.

La evaluación es validación cruzada estratificada de cinco particiones con predicciones *out-of-fold*: cada flujo se predice una única vez, por el modelo que no lo vio durante el entrenamiento.

El clasificador es el mismo en todas las comparaciones de este capítulo: un perceptrón multicapa de dos capas ocultas, de 256 y 128 unidades, precedido de un escalado estándar que se ajusta únicamente sobre la partición de entrenamiento. La sección 5.1 lo describe junto con el resto de modelos del trabajo. Mantenerlo fijo es deliberado, porque lo que aquí se compara son las representaciones y no las arquitecturas: si el modelo cambiase entre vistas, la diferencia medida no sería atribuible a la información que cada vista aporta. Las arquitecturas se comparan entre sí en el capítulo 5, y allí lo que se mantiene fijo es la vista.

Hay un detalle del submuestreo que afecta a la reproducibilidad. En el día del SSH el balanceo elige 358 flujos de ataque de entre 94.207, y cuáles se elijan mueve el resultado en la tercera cifra decimal, que es justo donde se separan las vistas. Fijar una semilla arbitraria y reportar ese número sería presentar como exacto algo que no lo es. Por eso cada comparación se repite diez veces con semillas distintas y se reporta la media.

Esas diez repeticiones no son diez semillas de la validación cruzada. Cada repetición extrae un subconjunto balanceado distinto y lo evalúa entero con validación cruzada de cinco particiones: el procedimiento son diez ejecuciones completas de la validación cruzada, es decir, cincuenta ajustes del modelo por vista y día. Lo que varía entre repeticiones es qué flujos de la clase mayoritaria entran en la comparación, que es la fuente de incertidumbre que importa cuando el balanceo descarta la mayor parte de esa clase, como aquí, que reduce los 94.207 flujos de ataque a los 358 benignos genuinos disponibles.

El algoritmo 2 recoge el procedimiento completo, porque descrito en prosa queda repartido entre varios párrafos y su reproducción exacta importa. Las dos fuentes de incertidumbre que el intervalo recoge corresponden a los dos bucles: el exterior sobre submuestreos y el interior sobre remuestreos.

Algoritmo 2 Protocolo de comparación entre vistas

Entrada: Flujos de ataque A y benignos genuinos B del servicio; vistas V ; repeticiones $R = 10$; remuestreos $N = 2000$

Salida: Macro-F1 medio e intervalo de confianza al 95 % por vista

```

1:  $n \leftarrow \min(|A|, |B|, 5000)$ 
2: para todo vista  $v \in V$  hacer
3:    $P_v \leftarrow \emptyset$ ;  $Q_v \leftarrow \emptyset$ 
4:   para  $r = 1$  hasta  $R$  hacer
5:      $S \leftarrow n$  flujos de  $A$  y  $n$  de  $B$ , al azar con semilla  $r$ 
6:      $\hat{y} \leftarrow$  predicción out-of-fold de  $S$  con 5-Fold estratificado
7:      $P_v \leftarrow P_v \cup \{\text{macro-F1}(y_S, \hat{y})\}$ 
8:     para  $b = 1$  hasta  $N/R$  hacer
9:        $I \leftarrow$  muestra con reemplazo de los índices de  $S$ 
10:       $Q_v \leftarrow Q_v \cup \{\text{macro-F1}(y_S[I], \hat{y}[I])\}$ 
11:    fin para
12:  fin para
13:  reportar media de  $P_v$  y percentiles 2.5 y 97.5 de  $Q_v$ 
14: fin para

```

Benigno genuino: el error que se cometió dos veces

La decisión más importante del protocolo es también la que más costó aprender. El tráfico benigno de referencia debe proceder de terceros, nunca del propio atacante fuera de la ventana etiquetada.

Parece obvio enunciado así, pero el error se cometió dos veces (secciones 3.6 y 4.2.3). En el día del SSH-Bruteforce, de los 2.022 flujos del puerto 22 etiquetados como benignos, 1.589 eran del propio atacante, conexiones suyas que caían fuera de los límites de la ventana, y solo 433 procedían de terceros. Con ese conjunto se estaba comparando ataque contra ataque, y el resultado era una medida sin sentido que además parecía razonable: ninguna vista superaba el 0.78 y se interpretó como el techo del problema. En el día del DoS-Hulk el mismo error afectaba al 65.1 % del benigno HTTP.

La corrección no consistió en filtrar esos flujos al evaluar, sino en arreglar la ventana y reetiquetarlos en el conjunto de datos, porque eran actividad del ataque. Tiene una consecuencia metodológica que declarar: el escenario contaminado ya no puede reconstruirse desde los datos actuales, de modo que las cifras que en su día lo describieron son históricas y no comparables con las de este capítulo.

Lo que generaliza es la regla. Hay que verificar que la ventana etiquetada cubre toda la actividad continua del atacante sobre el conjunto completo, y no sobre una captura suelta. Un artefacto de etiquetado no se detecta mirando las métricas, porque las del escenario contaminado eran perfectamente plausibles, sino auditando de dónde procede cada flujo de la clase negativa.

Métricas

Se reportan macro-F1 y *recall* de la clase de ataque. La primera resume el equilibrio entre precisión y sensibilidad sin dejarse arrastrar por la clase mayoritaria. La segunda

responde a la pregunta operativa de cuántos ataques se escapan, y se reporta siempre por separado porque un macro-F1 de 0.33 puede corresponder tanto a un clasificador mediocre como a uno que no detecta absolutamente nada.

Ambas van acompañadas de un intervalo de confianza al 95 % obtenido por *bootstrap* de 2.000 remuestreos repartidos entre las diez repeticiones del submuestreo, de modo que el intervalo recoge las dos fuentes de incertidumbre: el tamaño finito del conjunto evaluado y qué flujos concretos entraron en él. Con 358 flujos benignos genuinos en el día del SSH y 203 flujos de ataque en el día web, un macro-F1 sin intervalo no permitiría distinguir una vista de otra.

La exactitud no se usa, por el motivo que ilustra la sección 4.2.1. En el capítulo 5 se añaden el AUC y el error de calibración: cuando se compara entre dominios, una métrica que dependa de un umbral fijo puede ocultar por completo lo que el modelo ha aprendido.

Las tres vistas

Se comparan tres representaciones del mismo flujo, siempre las mismas:

- *Payload*: histograma de los 256 valores de byte de la carga útil agregada de la conexión, más su entropía normalizada.
- Metadatos de flujo: cuatro escalares de la conexión, que son el número de paquetes, los bytes totales, la longitud de la secuencia y la duración, con $\log(1 + x)$ para las colas pesadas.
- Conducta: los metadatos anteriores más tres características de ráfaga, el número de conexiones que el mismo origen ha abierto en el segundo, los cinco segundos y los treinta segundos previos, calculadas de forma causal, mirando solo al pasado, como haría un detector en línea.

Que sean siempre las mismas no es una obviedad. Una versión anterior de este trabajo comparaba la vista conductual en un día contra los metadatos básicos en otro, ambas rotuladas igual, y eso exageraba la diferencia entre regímenes. Sobre ese error se vuelve en la sección 4.2.2.

4.2. Los cuatro regímenes: dónde vive la firma

Antes de medir, describir. Los seis días cubren cuatro maneras cualitativamente distintas de que un ataque se haga visible en el tráfico. Esta sección las caracteriza inspeccionando los datos, sin clasificador de por medio: son hechos sobre los ataques, no sobre los modelos.

4.2.1. Régimen cifrado: la firma vive en la repetición

El día Wednesday-14-02-2018 contiene un ataque de fuerza bruta contra SSH: 94.207 flujos de ataque desde 13.58.98.64 contra 172.31.69.25:22, entre las 14:01 y las 15:33 hora local, sobre un total de 2.202.231 flujos.

El tráfico va cifrado y los bytes que transporta son, por diseño, indistinguibles del ruido: la entropía de la carga útil es de aproximadamente 7.4 tanto en el ataque como en el tráfico

legítimo. Lo que sí distingue al ataque es que se repite, y la figura 4.2 muestra hasta qué punto: la mediana de conexiones por origen en ventanas de cinco segundos es de 86 para el atacante frente a 1 para el tráfico legítimo. Un ataque de fuerza bruta es, por definición, un patrón de repetición.

El espejismo del 99.96 %. Este día sirve además para mostrar por qué la exactitud no aparece como métrica en el resto del capítulo. Un primer modelo entrenado sobre el histograma de bytes y la entropía alcanzaba una exactitud del 99.95 % en validación cruzada y del 99.96 % sobre el conjunto de prueba. Auditado contra el tráfico SSH benigno, que es el único que un detector desplegado encontraría a diario, su tasa de falsos positivos era del 96 %. El modelo no había aprendido a reconocer el ataque: había aprendido a reconocer el protocolo SSH, que ese día aparecía casi siempre asociado al ataque. Las dos cifras son correctas y son inútiles.

La figura 4.1 pone las dos juntas, que es la única manera de verlo. Por separado, la primera parece un resultado excelente y la segunda un modelo distinto.

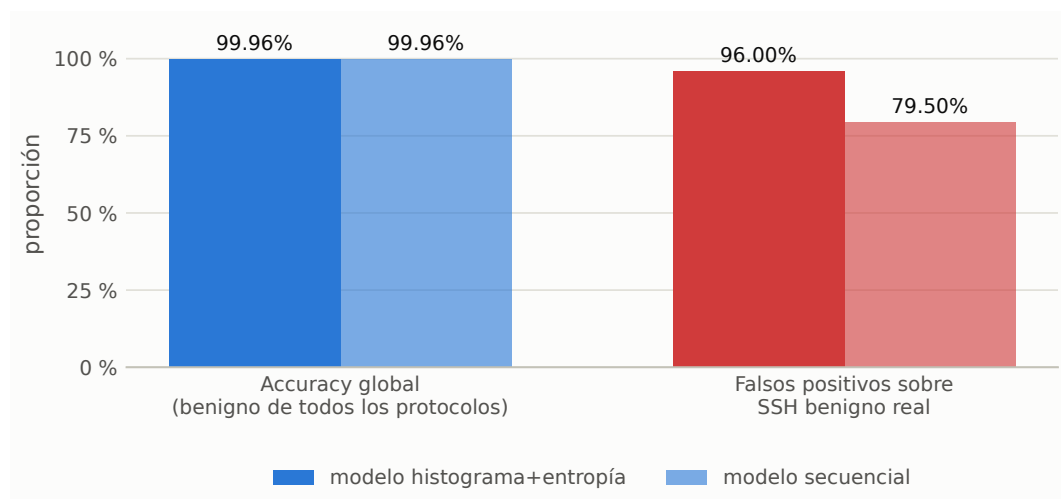


Figura 4.1: El mismo modelo, medido de dos maneras. A la izquierda, la exactitud sobre un conjunto benigno compuesto en su práctica totalidad por otros protocolos. A la derecha, los falsos positivos sobre tráfico SSH benigno real, que es el que un detector desplegado encontraría.

4.2.2. Régimen en claro: la firma vive en los bytes

El día Thursday-22-02-2018 contiene ataques web contra una aplicación DVWA: 203 flujos repartidos en fuerza bruta (142), XSS (42) e inyección SQL (19), desde 18.218.115.60 contra 172.31.69.28:80.

Aquí el contenido malicioso está literalmente en los bytes: las cadenas `union select`, `<script>` o la repetición de `POST login.php` son el ataque, no un rastro de él. Y la conducta, que en el SSH era la señal más nítida, aquí no existe: el contraste de ráfaga es de 1 conexión del ataque frente a 3 del benigno, es decir, el ataque abre menos conexiones que el tráfico normal. Es un ataque de bajo volumen.

El tipo de ataque vive en la secuencia, no en la composición

El día web permite además una pregunta que los otros no admiten, porque es el único con varios tipos de ataque distintos sobre el mismo servicio. En lugar de separar ataque de benigno, se puede pedir al modelo que distinga entre los tres tipos: fuerza bruta (142 flujos), XSS (42) e inyección SQL (19).

El resultado, en la tabla 4.1, es el único de todo el trabajo en que la representación del contenido importa por sí misma. La secuencia de bytes identifica el tipo con un macro-F1 de 0.9505, mientras el histograma de los mismos bytes se queda en 0.7462 y los metadatos de flujo en 0.7695.

Tabla 4.1: Macro-F1 *out-of-fold* al distinguir entre los tres tipos de ataque del día web, y *recall* de la clase minoritaria, que es la inyección SQL con 19 flujos.

Vista	Macro-F1	Recall SQLi
Payload, histograma	0.7462	0.2632
Payload, secuencia	0.9505	0.8421
Metadatos de flujo	0.7695	0.2632

La diferencia se concentra en la clase minoritaria. El histograma y los metadatos recuperan el 26.3 % de las inyecciones SQL, es decir, cinco de diecinueve; la secuencia recupera el 84.2 %, dieciséis de diecinueve. El motivo es que la inyección SQL y el ataque de fuerza bruta usan aproximadamente los mismos caracteres, y su composición de bytes es casi idéntica y solo el orden los distingue. La cadena `union select` no es un conjunto de letras, es una secuencia.

Es un resultado modesto por el tamaño de la muestra, con diecinueve ejemplos en la clase minoritaria, pero vale la pena retenerlo porque matiza todo el capítulo: cuando la comparación entre vistas satura, la comparación entre *representaciones de la misma vista* todavía puede discriminar.

4.2.3. Régimen volumétrico: la firma vive en el agregado

Tres días cubren este régimen, y juntos muestran algo que ninguno enseñaría por separado: el contraste de ráfaga se desploma tres órdenes de magnitud a medida que el ataque se distribuye.

Tabla 4.2: Mediana de conexiones abiertas por un mismo origen en ventanas de cinco segundos, para el ataque y para el tráfico legítimo del mismo día.

Día y ataque	Ataque	Benigno	Contraste
16-02 DoS-Hulk (1 origen)	11.626	3	$\times 3.875$
15-02 GoldenEye + Slowloris (1+1)	104	3	$\times 35$
20-02 DDoS-LOIC (10 orígenes)	39	3	$\times 13$
14-02 SSH-Bruteforce	86	1	$\times 86$
02-03 Bot (<i>beaconing</i>)	10	2	$\times 5$
22-02 Web (BF / XSS / SQLi)	1	3	$\times 0$

La lectura de la tabla 4.2 es que la ráfaga no es una propiedad de los ataques volumétricos sino de los ataques concentrados. Un DDoS repartido entre diez orígenes diluye por construcción la señal que un DoS de un solo origen exhibe de forma masiva. El día web cierra el rango por el otro extremo, con contraste nulo.

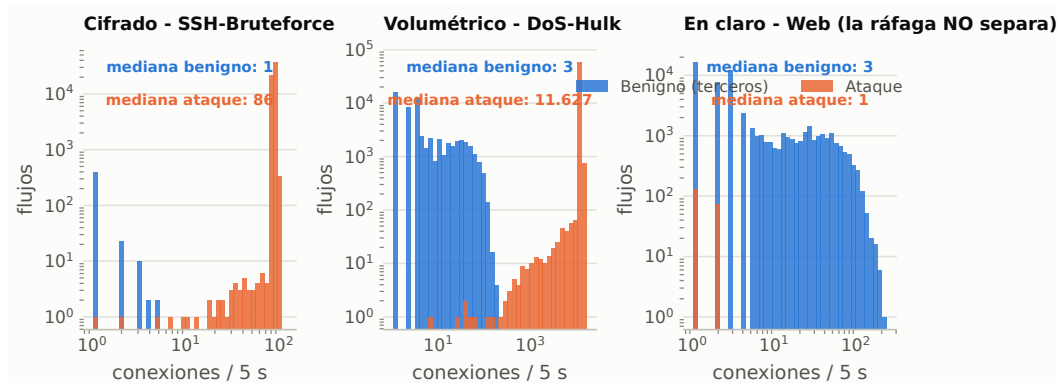


Figura 4.2: Conexiones abiertas por un mismo origen en ventanas de cinco segundos, en el día del SSH-Bruteforce. La mediana del atacante es 86; la del tráfico legítimo, 1.

4.2.4. Régimen de periodicidad: la firma vive en el intervalo

El día Friday-02-03-2018 contiene la actividad de una botnet (Ares): 142.921 flujos. Su firma no es el volumen, ya que su contraste de ráfaga es de apenas $\times 5$, sino la regularidad temporal. La figura 4.5 lo mide. El *beacon* late con dos intervalos discretos, en torno a 0.5 y 2.0 segundos, que concentran el 91.1 % de sus intervalos entre conexiones en dos bandas de 50 milisegundos. El tráfico benigno del mismo día es una curva suave decreciente: en la banda de 0.50 a 0.55 segundos cae el 41.8 % del ataque frente al 1.3 % del benigno, un contraste de $\times 32$.

Es el régimen que motivó una vista adicional, porque ninguna de las tres vistas del protocolo calcula el intervalo entre conexiones. Sobre ello vuelve la sección 4.7.

4.3. La medición uniforme: todo satura

Descritos los regímenes, la pregunta natural es cuál de las tres vistas gana en cada uno. La tabla 4.3 la responde, y la respuesta es que la pregunta no se puede contestar con estos datos.

El *payload* no baja de 0.9734 en ningún día y es el mejor o empata en cuatro de los seis. La conducta no baja de 0.9823. La única vista que falla de forma apreciable son los metadatos de flujo básicos, y solo en tres días. La figura 4.3 lo muestra como lo que es: una rejilla uniformemente alta, sin contraste que leer.

Vale la pena decir con claridad lo que esto derriba, porque era el marco con el que se empezó este trabajo. La hipótesis inicial era que cada régimen escondería su firma en una vista distinta, con lo que ninguna ganaría en todos y el enfoque híbrido quedaría justificado por esa vía. Medido con una sola métrica y un solo protocolo, ese marco no se sostiene: en el régimen volumétrico, donde se esperaba que ganase el agregado, gana el *payload*; y en el

Tabla 4.3: Macro-F1 *out-of-fold* de las mismas tres vistas, con el mismo protocolo y la misma métrica, en los seis días. Media de diez submuestreos.

Día y régimen	Payload	Metadatos	Conducta
14-02 cifrado	0.9842	0.9906	0.9985
22-02 en claro	0.9734	0.9186	0.9823
16-02 volumétrico	1.0000	0.9715	0.9999
15-02 volumétrico	0.9998	0.9621	0.9967
20-02 vol. diluido	1.0000	0.9987	0.9984
02-03 periodicidad	0.9990	0.9981	0.9983

Cifrado SSH-Bruteforce (14-02)	0.9842 <i>tras evasión: 0.3538</i>	0.9906 <i>invariante</i>	0.9985 <i>invariante</i>
En claro Web BF/XSS/SQLi (22-02)	0.9734 <i>tras evasión: 0.8852</i>	0.9186 <i>invariante</i>	0.9823 <i>invariante</i>
Volumétrico DoS-Hulk (16-02)	1.0000 <i>tras evasión: 0.0417</i>	0.9715 <i>invariante</i>	0.9999 <i>invariante</i>
Volumétrico GoldenEye+Slowloris (15-02)	0.9998 <i>no probada</i>	0.9621 <i>invariante</i>	0.9967 <i>invariante</i>
Volumétrico diluido DDoS-LOIC (20-02)	1.0000 <i>tras evasión: 0.0108</i>	0.9987 <i>invariante</i>	0.9984 <i>invariante</i>
Periodicidad Bot / Ares (02-03)	0.9990 <i>tras evasión: 0.0025</i>	0.9981 <i>invariante</i>	0.9983 <i>invariante</i>
	Payload (bytes)	Metadatos per-flujo	Conducta (meta + ráfaga)

Figura 4.3: La misma medición de la tabla 4.3, con lo único que distingue unas celdas de otras escrito debajo de cada una: cuánto sobrevive de esa vista cuando el atacante camufla los primeros bytes del flujo.

día de la botnet, donde se había afirmado que ninguna vista mide la firma, las tres pasan de 0.998.

La única diferencia real

Lo único que se mueve de forma estadísticamente sólida al cambiar de régimen es el desplome de los metadatos per-flujo. La figura 4.4 lo aísla: entre el día cifrado y el día en claro, el *payload* y la conducta apenas se mueven y sus intervalos se solapan, mientras los metadatos caen 7.2 puntos y son la única diferencia cuyos intervalos no se solapan.

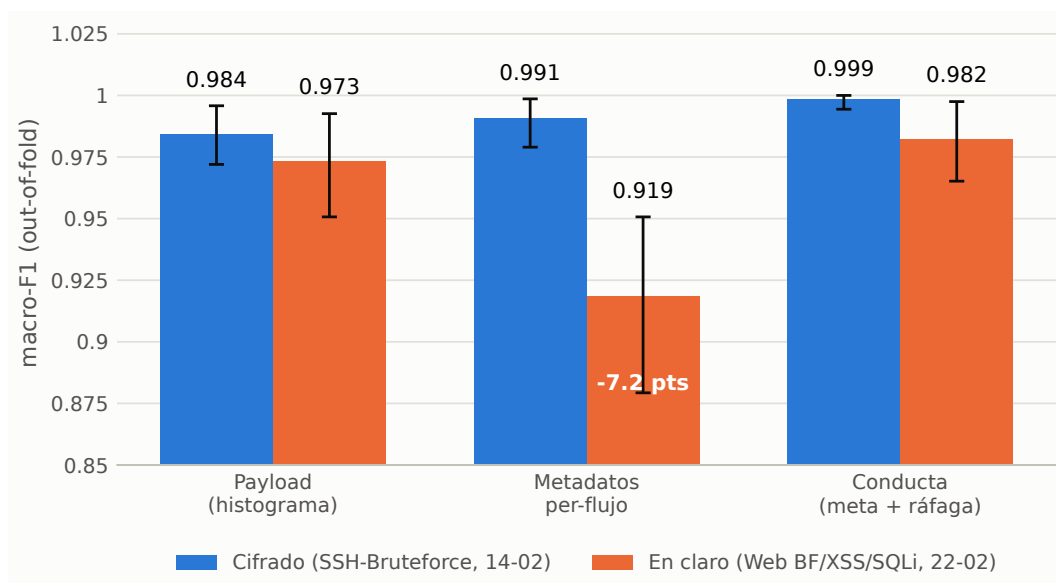


Figura 4.4: Las mismas tres vistas en los dos regímenes opuestos, con barras de error al 95 %.

Merece una nota de honestidad metodológica, porque una versión anterior de esta figura exageraba el efecto. Comparaba la vista conductual en el lado del SSH contra los metadatos básicos en el lado web, ambas rotuladas «metadatos», simplemente porque el día web nunca se había evaluado con la vista conductual. La caída aparente era de 1.000 a 0.929, y buena parte de esa caída no era el cambio de régimen sino el cambio de características. Medido como es debido, el efecto sigue existiendo y sigue siendo el único significativo, pero es la mitad de grande.

4.4. Por qué satura: una hipótesis y su refutación

Un resultado tan uniforme pide una explicación. La primera que se propuso en este trabajo resultó ser falsa, y merece contarse junto con el test que la tumbó, porque el mecanismo correcto solo aparece al descartarla.

La hipótesis: el atajo es la identidad del host

Cada uno de estos días contiene un único host atacante, con la excepción del DDoS del 20 de febrero, que tiene diez. Ese host genera decenas o centenares de miles de flujos que comparten todo lo que un host comparte consigo mismo: la misma pila TCP, el mismo cliente, las mismas cabeceras y el mismo ritmo.

Bajo esas condiciones cabe sospechar que la tarea que se le plantea al clasificador no es distinguir un ataque del tráfico normal sino, en la práctica, distinguir las conexiones de ese host de las de los demás. Esa tarea es fácil desde cualquier representación, lo que explicaría la saturación. Sería el fenómeno que Engelen y sus coautores describen para CIC-IDS2017 [7] bajo el nombre de *shortcut learning*: el modelo aprende un atajo correlacionado con la etiqueta en lugar del concepto que la etiqueta pretende nombrar.

La sospecha era razonable, y afectaba especialmente a la vista conductual, porque las características de ráfaga cuentan conexiones por origen y con un único atacante podrían estar funcionando como un identificador de ese host más que como una descripción de su conducta.

El test: apartar un atacante entero

La hipótesis es contrastable, pero no en un día con un solo atacante. El DDoS del 20 de febrero sí lo permite, porque tiene diez hosts atacantes con unos 29.000 flujos cada uno. Si el éxito se debiera a memorizar el host, evaluar sobre un atacante nunca visto debería hundirlo.

El protocolo aparta en cada pliegue un host atacante completo y, además, un grupo disjunto de hosts benignos, para que tampoco la clase negativa pueda memorizarse por host. Se entrena con 5.000 flujos por clase y se evalúa sobre 2.000, y se compara contra un control de exactamente los mismos tamaños con partición aleatoria por flujo: sin él, una caída podría deberse al tamaño del entrenamiento y no al cambio de host.

Se añade un segundo control, este positivo: una cuarta vista deliberadamente tramposa formada por los cuatro octetos de la dirección IP de origen, que no puede sino memorizar el host. Si el protocolo es sensible al efecto que pretende medir, esa vista debe acertar en la partición aleatoria y desplomarse al apartar un atacante. Sin esa comprobación, un resultado negativo no significaría nada.

El algoritmo 3 lo detalla, incluidos los dos controles.

Tabla 4.4: Macro-F1 al apartar un host atacante completo, frente a un control aleatorio de idéntico tamaño. Media y desviación sobre los diez pliegues. La última fila no es una vista candidata: es un identificador de host puro, incluido para verificar que el test detecta la memorización cuando existe.

Vista	Control	Atacante nuevo	Dif.
Payload (histograma + entropía)	1.0000 $\pm$ 0.0000	1.0000 $\pm$ 0.0000	+0.0000
Metadatos de flujo	0.9991 $\pm$ 0.0005	0.9992 $\pm$ 0.0006	+0.0001
Conducta (metadatos + ráfaga)	0.9986 $\pm$ 0.0006	0.9990 $\pm$ 0.0007	+0.0004
[control] octetos de la IP	0.9999 $\pm$ 0.0001	0.9332 $\pm$ 0.2000	-0.0667

El resultado: la hipótesis era falsa

La tabla 4.4 no deja margen. Apartar un host atacante entero no degrada ninguna de las tres vistas: las diferencias son de +0.0000, +0.0001 y +0.0004, es decir, nulas. Lo que las vistas aprenden no es la identidad del host.

El control positivo merece leerse con cuidado, porque su media esconde el dato interesante. Cae de 0.9999 a 0.9332, pero con una desviación de 0.2000: el efecto no está

Algoritmo 3 Leave-one-attacker-out con control negativo y positivo

Entrada: Hosts atacantes H_A ; hosts benignos H_B ; vistas V más la vista tramposa de octetos de la IP

Salida: Macro-F1 por vista, apartando un atacante, y su control aleatorio

- 1: Repartir H_B en $|H_A|$ grupos disjuntos al azar
- 2: **para** $i = 1$ **hasta** $|H_A|$ **hacer**
- 3: $test \leftarrow$ 2000 flujos del atacante h_i y 2000 del grupo benigno i
- 4: $train \leftarrow$ 5000 flujos de $H_A \setminus \{h_i\}$ y 5000 de los demás grupos
- 5: **para todo** vista $v \in V$ **hacer**
- 6: entrenar en $train$, predecir $test$, anotar macro-F1
- 7: **fin para**
- 8: **fin para**
 - ▷ Control negativo: mismos tamaños, partición aleatoria por flujo
- 9: **para** $i = 1$ **hasta** $|H_A|$ **hacer**
- 10: $train, test \leftarrow$ muestra aleatoria de igual tamaño, ignorando el host
- 11: **para todo** vista $v \in V$ **hacer**
- 12: entrenar, predecir y anotar macro-F1
- 13: **fin para**
- 14: **fin para**
 - ▷ Control positivo: la vista de octetos debe hundirse en el bucle 1

repartido sino concentrado en un solo pliegue. El motivo es que nueve de los diez atacantes tienen direcciones en el rango 18.2xx.x.x y el tráfico benigno está en 172.31.x.x, así que apartar un atacante deja otros nueve del mismo rango y el primer octeto sigue separando. Es la fuga de direcciones IP del capítulo 5, medida aquí a nivel de subred y no de host.

El pliegue decisivo es el de 52.14.136.135, la única atacante fuera de ese rango. Ahí el control positivo se hunde a 0.3333, es decir, predice una sola clase, mientras las tres vistas reales siguen dando 1.0000, 0.9987 y 0.9990. En el único pliegue donde se demuestra que un atajo por identidad de host falla, las vistas reales aguantan intactas. El test es sensible, y aun así no detecta memorización de host en ninguna de ellas.

Lo que sí explica la saturación

Descartado el host, queda la herramienta. Los diez atacantes de ese día ejecutan el mismo programa, y todos los ataques de todos los días de este conjunto están generados por una única herramienta cada uno. Lo que las vistas aprenden es esa herramienta y las condiciones de la captura, no el ataque como categoría.

La confirmación viene del mismo día por otra vía. Camuflar los primeros bytes del flujo hunde el *payload* de 1.0000 a 0.0108 (sección 4.5). Los dos experimentos se complementan de forma precisa: el leave-one-attacker-out dice que no es el host, y la evasión dice que es la herramienta.

El alcance de este test es limitado. Este test se puede hacer en un solo día de los seis, porque es el único con más de un atacante, y en los otros cinco la hipótesis del host sigue siendo, estrictamente, no contrastada. Y como los diez atacantes comparten herramienta, el test distingue memorizar el host de aprender la herramienta o la conducta, pero no puede

distinguir aprender la herramienta de aprender el ataque. Para eso hacen falta la evasión y la transferencia entre dominios.

Una reserva que el test levanta. La sospecha sobre la vista conductual, que la ráfaga fuese un identificador de host disfrazado, queda desmentida con datos: medida contra un atacante que nunca vio, la conducta rinde igual que contra atacantes conocidos, con 0.9990 frente a 0.9986. Cuenta conexiones por origen, sí, pero lo que captura es un patrón de comportamiento que se repite en hosts distintos.

Qué se puede afirmar y qué no. De la tabla 4.3 se sigue que las tres vistas contienen información suficiente para separar estos ataques de este tráfico benigno. No se sigue que el problema esté resuelto, ni que una vista sea mejor que otra, ni que un detector entrenado así funcionaría en una red real. Un macro-F1 de 0.99 en este montaje no mide capacidad de detección.

Un resultado anterior que queda explicado. Durante el trabajo se contrastaron varias arquitecturas del estado del arte, un *transformer* sobre bytes y una red de atención cruzada, y ninguna mejoró los resultados, algunas a un coste computacional entre dos y cincuenta veces mayor. En su momento se interpretó como un límite de los datos. La tabla 4.3 lo explica mejor: no mejoraban porque no quedaba nada que mejorar. El cuello de botella nunca estuvo en la capacidad del modelo.

4.5. Aciertan, pero por el motivo equivocado

Si las tres vistas aciertan, la pregunta útil deja de ser cuánto aciertan y pasa a ser por qué. Ahí sí se separan, porque no todas aciertan por la misma razón ni con la misma solidez.

El payload cifrado lee el banner, no el cifrado

El 0.9842 del *payload* en el día del SSH no procede de leer contenido cifrado, que sería imposible. Procede del *banner* en claro que precede a la negociación de la sesión y que identifica al cliente. El atacante emite dos banners distintos, y el modal es literalmente SSH-2.0-paramiko_2.0.0, frente a los 81 de los clientes legítimos como PuTTY, libssh2 u OpenSSH. El análisis de saliencia lo confirma: el 31.1 % de la importancia atribuida se concentra en los primeros 48 bytes, la zona del banner, y no en el cuerpo cifrado.

Es una firma real, pero de la herramienta y no del ataque. El mismo patrón se repite en los días volumétricos, donde lo que el modelo aprende son las cabeceras y los *User-Agent* característicos de Hulk, GoldenEye y LOIC.

La prueba: camuflar los primeros bytes

La forma de comprobarlo es directa. Si el modelo se apoya en los primeros bytes del flujo, sobrescribirlos con los de un cliente legítimo debería hundirlo, y a un atacante real le costaría muy poco hacerlo.

Eso es exactamente lo que ocurre. El experimento completo se desarrolla en la sección 5.3, tabla 5.1, porque pertenece al capítulo de las perturbaciones. Basten aquí las dos cifras de

titular: el *recall* del *payload* en el día de la botnet cae de 1.0000 a 0.0025, y en el DDoS a 0.0108. El *payload* deja de funcionar mientras el ataque sigue funcionando igual de bien, porque el atacante no ha cambiado su técnica, solo su saludo.

La excepción confirma la regla, y hubo que medirla dos veces. El día web es el único que aguanta, y el camino hasta esa cifra merece contarse. El primer experimento sobrescribía 160 bytes y el *recall* se desplomaba a 0.0164, lo que parecía confirmar que también allí la firma era una huella de herramienta. Al inspeccionar los flujos modificados quedó claro que el experimento estaba mal planteado: en 160 bytes cabe la inyección entera, de modo que sobrescribirlos no evade la detección, destruye el ataque. Repetido con un prefijo de 24 bytes, que no llega a la carga maliciosa, el *payload* aguanta en 0.8852. Su firma sí es contenido genuino.

La conducta es invariante, y no por mérito. Las vistas de metadatos y de conducta no cambian un solo valor bajo este ataque, sencillamente porque no leen el contenido. Es una ventaja estructural, no una virtud del modelo, y hay que presentarla como tal: dice que esta forma concreta de evasión no las afecta, no que sean robustas frente a cualquier evasión. Un atacante que ralentice deliberadamente su fuerza bruta ataca la ráfaga igual de barato.

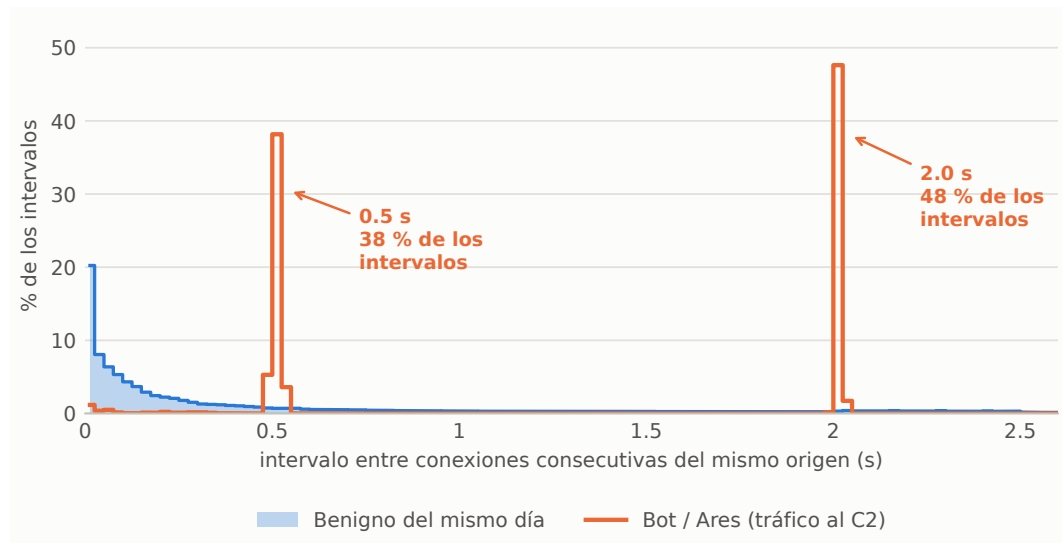


Figura 4.5: Distribución de los intervalos entre conexiones consecutivas del mismo origen. El ataque concentra el 91.1 % de sus esperas en dos bandas de 50 ms; el tráfico benigno del mismo día es una curva suave.

4.6. El límite del enfoque: lo que el payload no puede ver

Los apartados anteriores comparan vistas que aciertan. Este trata de los casos en que una de ellas no tiene nada que mirar, que es una situación cualitativamente distinta de acertar poco.

Un ataque sin payload en el propio día del SSH

El día 14 de febrero contiene un segundo ataque, de fuerza bruta contra FTP, que no aparece en ninguna tabla de este capítulo porque no genera un solo flujo con carga útil analizable. El atacante (18.221.219.4) dirige 162.801 conexiones a 172.31.69.25:21, de las cuales 134.945 quedan en estado REJ con cero bytes, porque el servicio FTP de la víctima estaba caído durante la captura y rechazó los intentos. Las restantes tampoco llegan a exportar datos de aplicación. El neto es cero flujos FTP con *payload*.

La causa es más concreta de lo que parece. Es una circunstancia de esta captura, un servicio caído, y no una propiedad del protocolo FTP. Lo que generaliza no es el mecanismo sino la consecuencia, y esa consecuencia reaparece por otras vías: el SlowHTTPTest del 16 de febrero también deja cero flujos con *payload*, en su caso porque un ataque de agotamiento lento mantiene conexiones abiertas sin llegar a intercambiar datos.

Infiltration: el 94.4 % de una intrusión sin un byte que analizar

El caso más severo es el día de la Infiltration, y llega por la vía del resultado negativo. Tras comprometer una máquina, el atacante ejecuta una fase de reconocimiento desde dentro de la red. De las 193.097 conexiones salientes de la víctima, 182.323, el 94.4 %, no llevan un solo byte de aplicación: 158.474 quedan en estado S0, que es un SYN sin respuesta, y 21.069 en REJ. Zeek no identifica servicio en 183.907 de ellas. Son un barrido de puertos contra 612 destinos.

La figura 4.6 reparte esas conexiones según lleven o no carga útil. El contraste dentro del mismo día es lo que cierra el argumento: las 5 conexiones al canal de mando y control (13.58.225.34:31337) sí llevan *payload*, con 195.816 bytes de aplicación y el 100 % de ellas con contenido, y son perfectamente visibles. El análisis de contenido ve la parte pequeña de la intrusión y es ciego a la grande.

No es que el *payload* funcione peor: es que no puede ver. Una fase entera de una intrusión real es estructuralmente invisible para un enfoque basado en contenido, y ninguna mejora del modelo lo arregla, porque no hay datos que mejorar. La literatura recoge el mismo problema al reetiquetar CIC-IDS2017 bajo la categoría «*Attempted*» [7] (sección 2.4).

Este resultado es, con diferencia, el argumento más fuerte a favor de combinar vistas, y es el único de todo el capítulo que no depende de una comparación de métricas.

4.7. Síntesis: la pregunta correcta

Este capítulo empezó con la intención de mostrar qué vista gana en cada régimen y termina mostrando que dentro del día no gana ninguna en particular, porque el montaje experimental no lo permite. Queda recoger qué sigue en pie.

Los cuatro regímenes son reales. La firma del SSH-Bruteforce vive en la repetición, con 86 conexiones por ventana frente a 1; la de los ataques web en los bytes; la del DoS en el agregado; y la de la botnet en el intervalo, con el 91.1 % de sus esperas en dos bandas de 50 ms. Eso se comprueba inspeccionando el tráfico y no depende de ningún clasificador.

Lo que no se sostiene es la asignación de una vista ganadora a cada régimen. Con un solo host atacante por día, cualquier vista dispone de un atajo suficiente, y las diferencias entre ellas caen dentro del ruido.

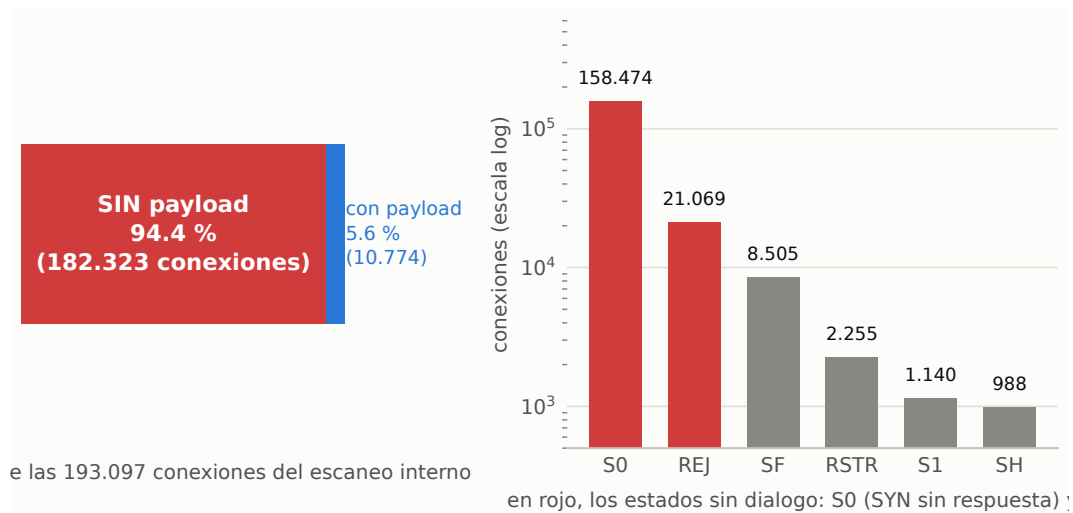


Figura 4.6: Reparto de las 193.097 conexiones salientes de la víctima según lleven o no carga útil de aplicación. El 94.4 % no lleva un solo byte que analizar.

Hay una excepción a la saturación, y apunta al capítulo siguiente. Ninguna de las tres vistas calcula el intervalo entre conexiones, que es donde vive la firma de la botnet. El día 02-03 se detecta igualmente al 0.999, por la huella HTTP del canal de mando, pero se detecta por el motivo equivocado, y por eso es también el día en que la evasión resulta más devastadora, de 1.0000 a 0.0025. La saturación y la fragilidad son dos caras de lo mismo.

La pregunta correcta es, por tanto, otra. Si dentro del día todas las vistas aciertan, lo que hay que preguntar no es cuál acierta más, sino cuál sigue acertando cuando cambian las condiciones: cuando el atacante camufla sus bytes, cuando el modelo se aplica a otro día o a otro conjunto de datos, y cuando hay que fijar un umbral de decisión sin ver antes los datos de destino. Ese es el contenido del capítulo 5.

Generalización, calibración y robustez

El capítulo anterior termina con una pregunta. Si dentro de un mismo día las tres vistas aciertan por encima de 0.97 y la comparación no las distingue, cómo se decide cuál usar. La respuesta de este capítulo es que hay que perturbar las condiciones y ver cuál sigue en pie.

Se aplican cuatro perturbaciones, de menor a mayor exigencia: mirar qué parte de la entrada usa realmente el modelo, dejar que el atacante modifique esa parte, aplicar el modelo a otro día del mismo régimen y aplicarlo a otro conjunto de datos. Bajo todas ellas las vistas dejan de ser equivalentes, y el orden en que caen no es el que sugerían sus métricas.

5.1. Modelos empleados

Antes de perturbar queda declarar qué se perturba. Este trabajo emplea diez modelos, y la distinción entre los propios y los tomados del estado del arte se mantiene explícita en toda la memoria.

Modelos propios

- Perceptrón multicapa de dos capas ocultas, de 256 y 128 unidades, con activación ReLU, optimizador Adam, regularización L_2 de 10^{-4} y parada temprana, precedido de un escalado estándar ajustado solo sobre la partición de entrenamiento. Es el clasificador que produce todas las comparaciones entre vistas del capítulo 4, y se emplea también sobre las representaciones tabulares del resto de la memoria. Su número de parámetros depende de la dimensión de entrada de cada vista, que va de 4 a 257, por lo que no entra en la comparación a presupuesto igualado que se describe más abajo.
- byte-CNN: red convolucional unidimensional sobre la secuencia cruda de bytes, con *kernels* de varios tamaños en paralelo (3, 5 y 7) para captar n -gramas de longitud distinta, seguidos de agrupación máxima global. 29.762 parámetros.

- byte-LSTM: red recurrente bidireccional sobre la misma entrada, pensada como contraste, porque si la firma fuese secuencial de largo alcance debería ganar. 52.482 parámetros.
- Híbrido de dos ramas con fusión tardía: una rama convolucional sobre los bytes y otra densa sobre los metadatos, concatenadas en el espacio de representaciones y no en el de entradas. 31.042 parámetros.
- Autoencoder e Isolation Forest para la detección no supervisada, entrenados únicamente con tráfico benigno.

Modelos del estado del arte, implementados para contrastar

- byte-Transformer: codificador con autoatención sobre la secuencia de bytes, con *token* [CLS] y codificación posicional aprendida, de 4 cabezas y 2 capas. 41.922 parámetros.
- CrossAttentionNet: fusión por atención cruzada bidireccional entre la rama de bytes y la de metadatos, en lugar de concatenación. 72.066 parámetros.
- Random Forest y XGBoost [9] sobre características de flujo, que son el estándar de facto en la literatura de detección de intrusiones tabular.

Los modelos clásicos se construyen sobre scikit-learn [23] y los profundos sobre PyTorch [24].

Por qué el presupuesto de parámetros importa

Todos los modelos secuenciales se comparan en una banda estrecha, entre 29.762 y 72.066 parámetros. No es un detalle de implementación. Sin esa restricción, una comparación entre arquitecturas mide sobre todo la capacidad: un *transformer* de diez millones de parámetros que superase a un byte-CNN de treinta mil no demostraría que la autoatención sea mejor para este problema, sino que un modelo mayor ajusta mejor, o peor si sobreajusta. Igualar el presupuesto es lo que permite atribuir la diferencia a la arquitectura, que es lo que se quiere medir.

5.2. Interpretabilidad: qué mira el modelo

La primera perturbación no es tal. Consiste en abrir el modelo y ver qué parte de la entrada usa, y es el paso que motiva todo lo demás.

Sobre el byte-CNN entrenado en el día del SSH-Bruteforce se calcula la saliencia por gradientes: la magnitud del gradiente de la salida respecto a cada byte de la entrada, promediada sobre los flujos de ataque. Mide cuánto cambiaría la decisión si ese byte cambiase.

El resultado, en la figura 5.1, es inequívoco. El 31.1 % de la importancia total se concentra en los primeros 48 bytes de un flujo que puede llegar a 64 kilobytes. Esa zona es exactamente donde vive el *banner* SSH en claro, que precede a la negociación de la sesión. El cuerpo cifrado, que ocupa el resto, apenas contribuye.

Inspeccionando esos bytes se ve por qué. El atacante emite dos banners distintos en todos sus flujos, siendo el modal literalmente SSH-2.0-paramiko_2.0.0, frente a los 81 banners distintos de los clientes legítimos, entre ellos OpenSSH, libssh2 y PuTTY. Y la entropía de la carga útil es de 7.4 en ambas clases, así que el contenido cifrado es literalmente indistinguible.

El modelo no ve el cifrado, ve la herramienta. Es una firma real, porque el atacante efectivamente usa paramiko, pero no es una firma del ataque sino del programa con que se ejecutó. Cualquier atacante que cambie de biblioteca la anula, sin modificar su técnica.

Este hallazgo es el que obliga a todo lo que sigue. Si un macro-F1 alto puede sostenerse sobre una firma así, la métrica por sí sola no informa de nada, y hay que someter cada vista a la pregunta de qué ocurre cuando esa firma desaparece.

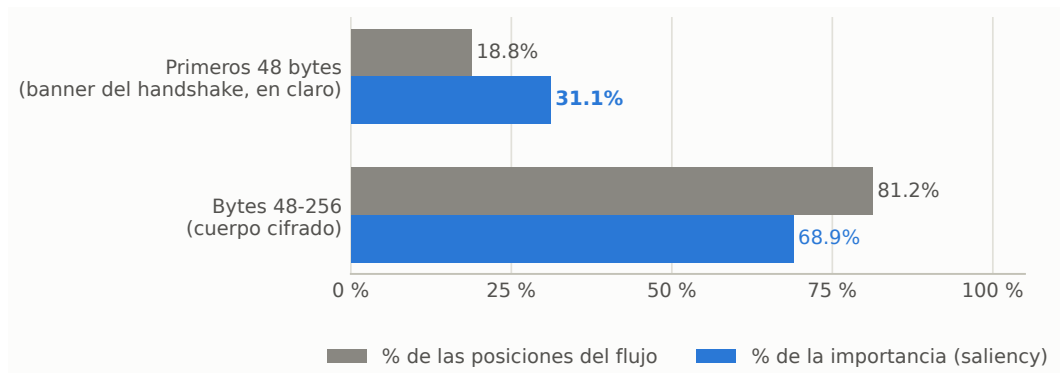


Figura 5.1: Importancia atribuida a cada posición de la entrada. El 31.1 % se concentra en los primeros 48 bytes, que es donde vive el *banner* en claro; el cuerpo cifrado apenas contribuye.

5.3. Robustez ante evasión

La forma directa de responder a esa pregunta es hacer lo que haría un atacante: sobrescribir los primeros bytes de cada flujo de ataque con los de un cliente legítimo, dejando intacto todo lo demás. Es una modificación barata, cambiar de biblioteca o falsear una cabecera, y no altera la técnica del ataque.

Una advertencia metodológica que costó un experimento

Un test de evasión solo es válido si el ataque sigue siendo el ataque después de la modificación. Es fácil pasarlo por alto, y en este trabajo se pasó.

El primer experimento sobrescribía 160 bytes en todos los días. En el día web el *recall* del *payload* se desplomaba de 0.9672 a 0.0164, lo que parecía demostrar que también allí la firma era una huella de herramienta. Inspeccionando los flujos modificados quedó claro que el resultado no significaba eso: en 160 bytes cabe la inyección SQL entera, de modo que sobrescribirlos no evade la detección, destruye el ataque. El modelo no fallaba, acertaba al no marcar como ataque un flujo que ya no lo era.

Repetido con un prefijo de 24 bytes, que no llega a la carga maliciosa, el *payload* aguanta en 0.8852. La conclusión se invierte por completo.

El resultado

La tabla 5.1 recoge el experimento corregido.

Tabla 5.1: *Recall* de la clase de ataque antes y después del camuflaje.

Día y ataque	Original	Camuflado
02-03 Bot (Ares)	1.0000	0.0025
20-02 DDoS-LOIC	1.0000	0.0108
16-02 DoS-Hulk	1.0000	0.0417
14-02 SSH-Bruteforce	1.0000	0.3538
22-02 Web (24 B)	0.9672	0.8852

La tabla recoge solo el análisis de contenido, porque es el único al que el camuflaje afecta: las vistas de metadatos y de conducta no leen un byte del contenido y sus valores no cambian. La figura 5.2 las incluye para verlo.

Cuatro de los cinco días se desploman. En el día de la botnet el *payload* pasa de detectarlo todo a detectar el 0.25 %, y en el DDoS al 1 %. El único que resiste es el día web, y resiste porque allí los bytes que el modelo usa son el ataque: `union select` no es un rastro de la inyección SQL, es la inyección.

Esto rescata un resultado que el capítulo anterior había dejado en suspenso. La comparación de macro-F1 no distinguía el día web de los demás, porque todos saturaban, pero la evasión sí lo distingue: es el único régimen en que el análisis de contenido detecta el ataque y no la herramienta. Lo que la métrica no separaba, la perturbación lo separa.

Invariante no es lo mismo que robusto

Las vistas de metadatos y de conducta no cambian un solo valor bajo este ataque, sencillamente porque no leen el contenido. Es lo que es: una ventaja estructural frente a esta forma concreta de evasión, no una virtud general. Un atacante que espacie deliberadamente sus conexiones ataca la ráfaga con el mismo esfuerzo, y este trabajo no ha medido ese caso. Afirmar que la conducta es robusta sin más sería el tipo de conclusión que el capítulo anterior desmonta.

5.4. Cómo combinar las dos ramas

Antes de seguir perturbando queda resolver una cuestión de diseño que el capítulo 2 planteaba sin datos: si se dispone de una vista de contenido y otra de flujo, dónde conviene unirlos.

Se comparan tres estrategias. La primera es la fusión ingenua, que concatena las entradas crudas y entrega el vector resultante a un único modelo. La segunda es la fusión tardía de dos ramas, en la que cada rama construye su representación por separado y la concatenación ocurre en el espacio de representaciones, justo antes de la capa de decisión. La tercera es cada rama por separado, como referencia.

La figura 5.3 muestra la segunda, que es la que este trabajo emplea, y señala dónde habría concatenado la primera.

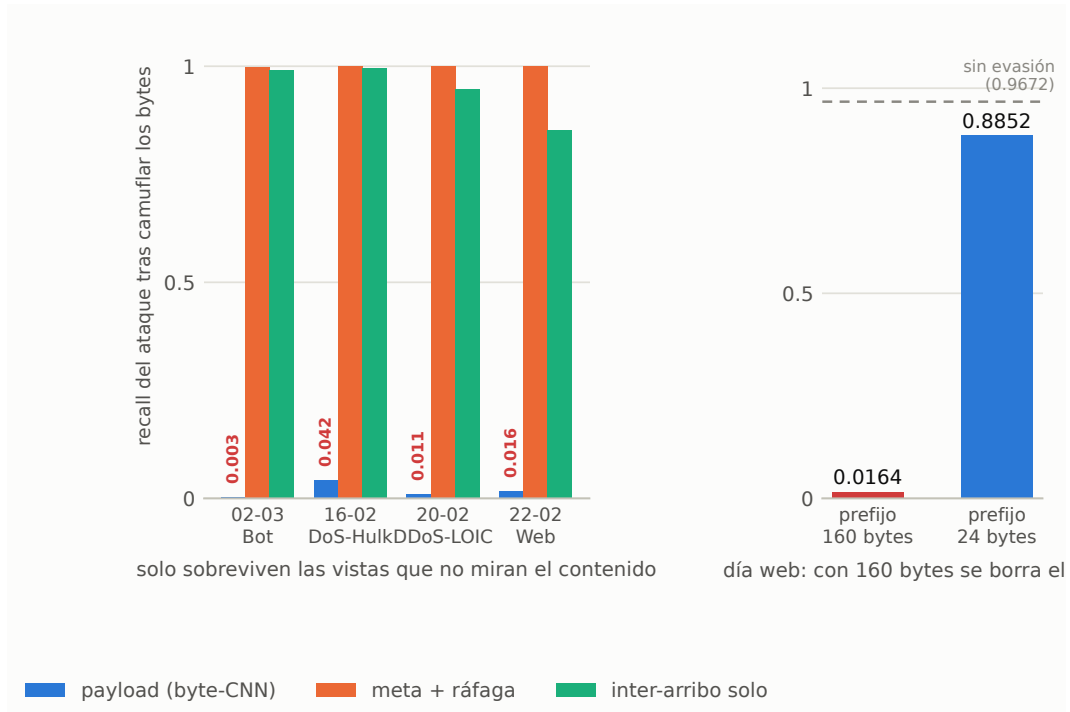


Figura 5.2: Recall de la clase de ataque tras sobrescribir los primeros bytes de cada flujo, por vista y por día. El panel derecho compara los dos tamaños de prefijo ensayados en el día web.

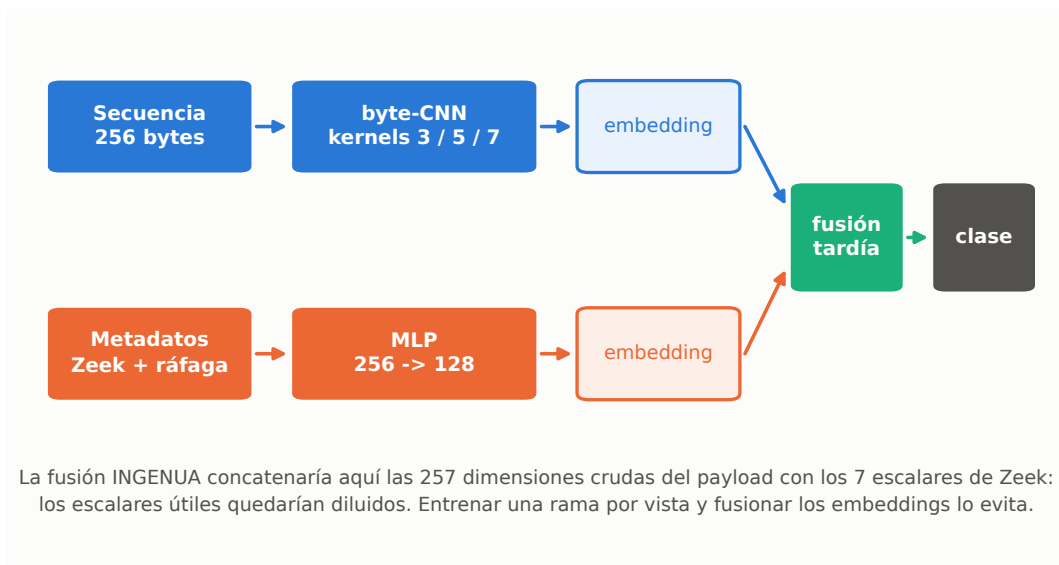


Figura 5.3: El híbrido de dos ramas con fusión tardía. Cada vista se convierte primero en su propia representación y la unión ocurre entre representaciones de dimensión comparable, no entre entradas crudas.

Tabla 5.2: Macro-F1 *out-of-fold* de cada rama por separado y de las dos estrategias de fusión, en los dos regímenes opuestos y en la tarea multitipo del día web.

Estrategia	14-02 SSH	22-02 web	22-02 multitipo
Solo payload (byte-CNN)	0.9988	1.0000	0.9679
Solo metadatos (MLP)	0.9931	0.9184	0.7695
Fusión ingenua (concatenar entradas)	0.9988	0.9803	0.9009
Fusión tardía (dos ramas)	1.0000	0.9975	0.9628

La tabla 5.2 deja ver el problema de la fusión ingenua. En el día web pasa de 1.0000, que es lo que consigue la rama de contenido sola, a 0.9803; y en la tarea multitipo, de 0.9679 a 0.9009. Fusionar las entradas hace el resultado peor que la mejor de sus partes.

El motivo es de escala. El histograma aporta 257 dimensiones, que son las 256 frecuencias de byte más la entropía, y los metadatos aportan cuatro escalares, de modo que al concatenarlos los segundos quedan diluidos y el modelo acaba comportándose como si solo tuviera la vista de contenido, con la desventaja añadida de un espacio de entrada mayor y peor condicionado. La fusión tardía evita el problema porque cada rama se entrena para producir una representación de dimensión comparable antes de unirlos.

Es un detalle de diseño con una consecuencia práctica: la ventaja del enfoque híbrido no se obtiene por el hecho de disponer de las dos fuentes, sino por unirlos en el sitio correcto. Un trabajo que concatene entradas y concluya que la fusión no aporta nada habrá medido la implementación, no la idea.

5.5. Detección sin etiquetas de ataque

Todo lo anterior supone un conjunto de entrenamiento con ataques etiquetados. Esa suposición es fuerte, porque un ataque nuevo, por definición, no está etiquetado en ninguna parte. La alternativa es la detección de anomalías: entrenar únicamente con tráfico benigno y marcar lo que se desvía.

Se ensayan dos métodos clásicos, un Isolation Forest y un autoencoder, sobre las mismas vistas y en tres regímenes. Se reporta el área bajo la curva ROC, porque en este planteamiento no hay umbral natural y la métrica debe ser independiente de él.

Tabla 5.3: Área bajo la curva ROC. IF es Isolation Forest y AE es autoencoder. El azar corresponde a 0.5.

Régimen	Payload (IF)	Payload (AE)	Conducta (IF)
Cifrado (14-02, SSH)	0.6946	0.5683	1.0000
En claro (22-02, web)	0.7580	0.7920	0.9121
Volumétrico (16-02, DoS)	0.3411	0.4791	0.5402

La tabla 5.3 dice tres cosas. La primera es que sin etiquetas de ataque la tesis del trabajo se replica: cada régimen esconde su anomalía en una vista distinta, y la conducta resuelve el cifrado con 1.0000 mientras el contenido se queda en 0.6946.

La segunda es que el régimen volumétrico es intrínsecamente difícil sin etiquetas. Las tres combinaciones quedan por debajo de 0.55, y el Isolation Forest sobre contenido baja

de 0.5, es decir, ordena peor que el azar. El motivo es que las peticiones que genera Hulk son sintácticamente más regulares que el tráfico web legítimo, y un detector de anomalías entrenado con lo normal las considera menos anómalas que el propio tráfico normal.

La tercera es una advertencia sobre las cifras del resto del capítulo. Los valores supervisados que aparecen en toda la memoria por encima de 0.97 dependen de disponer de ataques etiquetados del mismo tipo. Cuando esa suposición se retira, el rendimiento cae entre veinte y cincuenta puntos según el régimen. Es la distancia real entre lo que se mide en un conjunto de datos etiquetado y lo que un detector encontraría ante una amenaza nueva.

5.6. Generalización entre días del mismo régimen

La segunda perturbación es cambiar de día. Se toman los tres días volumétricos del 2018, que son el 16-02, el 15-02 y el 20-02, y se forman los seis pares dirigidos posibles: se entrena en uno y se evalúa en otro, con el mismo régimen de ataque pero distinta herramienta, distinta víctima y distinto tráfico de fondo.

Una lección metodológica antes que un resultado

Este hallazgo se enunció tres veces y cambió las tres. Merece contarse así porque la lección vale tanto como el número.

La primera versión, con dos días, afirmaba que el modelo más expresivo generaliza peor, porque el byte-CNN colapsaba y el histograma no. La segunda, al añadir el tercer día y completar los seis pares, mostró que la regla era falsa: se había inducido una ley con dos observaciones. La tercera llegó al cambiar de métrica, y mostró que lo que colapsaba no era lo que parecía.

El diagnóstico: no falla la representación, falla el umbral

Con umbral fijo en 0.5, el byte-CNN da un macro-F1 de 0.3333 en cuatro de los seis pares. Ese valor no es simplemente malo: es exactamente lo que produce un clasificador que asigna todo a una sola clase. Su *recall* de ataque es 0.0000. Leído así, el modelo no transfiere en absoluto.

El AUC dice otra cosa. La tabla 5.4 lo pone al lado.

Tabla 5.4: Entrenar en un día volumétrico y evaluar en otro. El macro-F1 con umbral fijo sugiere colapso; el AUC muestra que el orden de las puntuaciones se conserva.

Par (entreno → prueba)	AUC-ROC	F1 (umbral 0.5)	F1 (umbral óptimo)
16-02 → 20-02	0.9724	0.0000	0.9849
20-02 → 16-02	0.9886	0.0000	0.9918
15-02 → 20-02	0.9758	0.0000	0.9872
20-02 → 15-02	0.9854	0.0000	0.9899
15-02 → 16-02	1.0000	1.0000	1.0000
16-02 → 15-02	1.0000	0.9581	0.9999

La figura 5.4 contrapone ambas lecturas. El AUC no baja de 0.9724 en ninguno de los seis pares. El modelo ordena correctamente los flujos del día nuevo, porque los de ataque reciben puntuaciones más altas que los benignos, y sin embargo, con el umbral heredado del día de entrenamiento, no acierta ni uno. La representación transfiere; lo que no transfiere es dónde cortar.

Es una distinción con consecuencias prácticas inmediatas, y es invisible para cualquier métrica que dependa de un umbral fijo. Un trabajo que hubiese reportado solo el F1 habría concluido que el byte-CNN no generaliza entre días, y habría estado equivocado.

5.6.1. De diagnóstico a receta: recalibración

Si el problema es el umbral, la solución es recalibrar. Se aplica escalado de Platt [5], que ajusta una función $\sigma(A \cdot \text{logit}(p) + B)$ sobre unas pocas muestras etiquetadas del día de destino, sin reentrenar el modelo. La pregunta operativa es cuántas etiquetas hacen falta, y la respuesta son veinticinco. La tabla 5.5 recoge los cuatro pares que colapsaban.

Tabla 5.5: Macro-F1 y error de calibración esperado (ECE) antes y después de aplicar escalado de Platt con 25 flujos etiquetados del día de destino. No se reentrena el modelo.

Par	Sin recalibrar		Con 25 etiquetas	
	F1	ECE	F1	ECE
16-02 → 20-02	0.3333	0.4973	0.9750	0.0216
20-02 → 16-02	0.3333	0.4991	0.9912	0.0087
15-02 → 20-02	0.3332	0.4967	0.9828	0.0170
20-02 → 15-02	0.3333	0.4993	0.9698	0.0216

La figura 5.5 contrapone las dos situaciones en los seis pares. El error de calibración cae de aproximadamente 0.499, que es el máximo posible y corresponde a un modelo sistemáticamente seguro y sistemáticamente equivocado, a menos de 0.022. Ampliar de 25 a 1.000 etiquetas apenas mejora nada, salvo en el par más difícil, donde el *recall* sube de 0.9571 a 0.9997.

Traducido a despliegue: cuando un detector entrenado en unas condiciones se lleva a otras, no hay que reentrenarlo, basta con que un analista etiquete veinticinco conexiones. Es una recomendación concreta, barata y verificable, y sale directamente del diagnóstico anterior.

El mismo fenómeno ha sido descrito de forma independiente en la literatura reciente [25], que lo corrige también con escalado de Platt. Que dos trabajos lleguen por separado al mismo diagnóstico refuerza que no es un artefacto de este montaje.

5.7. Una vista para la periodicidad, y qué la hace funcionar

El capítulo 4 deja una promesa abierta. La firma de la botnet vive en el intervalo entre conexiones, y ninguna de las tres vistas del protocolo lo calcula: la ráfaga cuenta cuántas conexiones hay en una ventana, no cada cuánto llegan. Esta sección cierra esa promesa añadiendo una cuarta vista y midiendo qué aporta.

5.7. Una vista para la periodicidad, y qué la hace funcionar

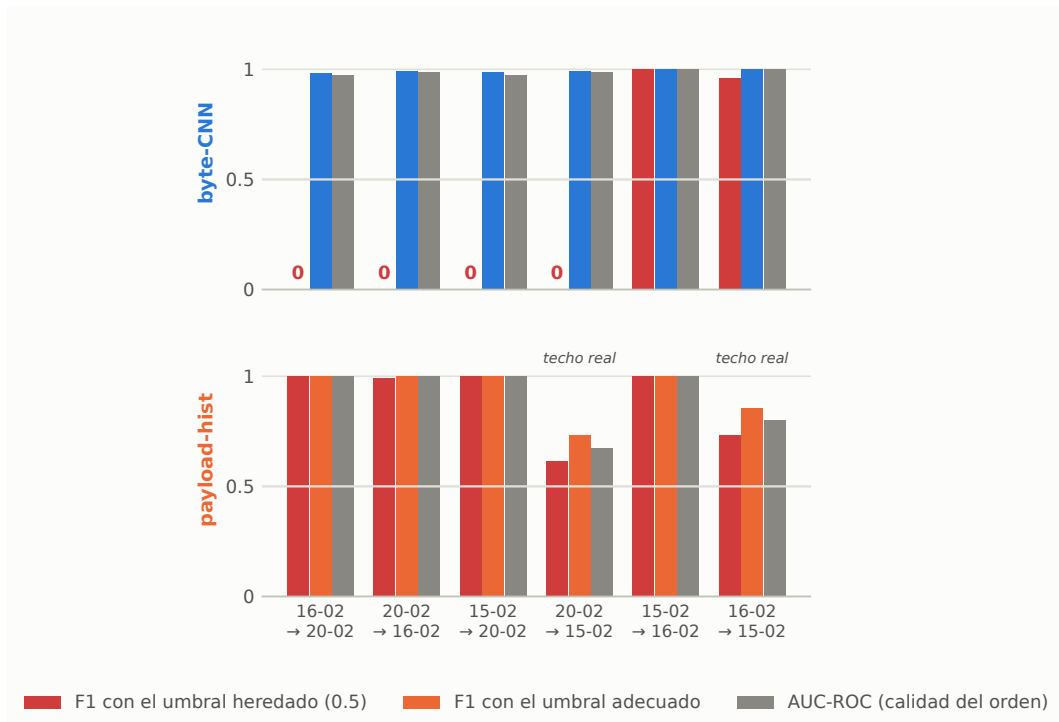


Figura 5.4: Para cada par de días, el macro-F1 con umbral fijo frente al AUC. El primero sugiere colapso; el segundo muestra que el orden de las puntuaciones se conserva.



Figura 5.5: Macro-F1 y error de calibración antes y después de recalibrar con 25 flujos etiquetados del día de destino, en los seis pares de días. Los cuatro pares que colapsaban se recuperan y el error de calibración cae de su valor máximo a casi cero.

La vista de inter-arribo son seis características por flujo, todas derivadas de los intervalos entre conexiones consecutivas del mismo origen: el intervalo anterior y el siguiente, la media y el coeficiente de variación en una ventana local, una medida de regularidad y la fracción de intervalos cercanos a la mediana. Se calculan desde el mismo fichero que las demás, sin volver a procesar las capturas.

Qué aporta como grupo

Tabla 5.6: Macro-F1 *out-of-fold* en el día de la botnet, combinando los metadatos de flujo con cada grupo temporal y con los grupos por separado.

Conjunto de características	Macro-F1
Metadatos de flujo	0.9985
Metadatos + ráfaga	0.9991
Metadatos + inter-arribo	0.9989
Metadatos + ráfaga + inter-arribo	0.9989
Inter-arribo solo (6 características)	0.9914
Ráfaga sola (3 características)	0.9909

La tabla 5.6 obliga a una conclusión modesta y hay que darla como tal. Sobre los metadatos, la ráfaga y el inter-arribo aportan prácticamente lo mismo, con 0.9991 frente a 0.9989, y combinarlos no mejora. Es decir, la vista nueva no supera a la que ya había.

Su interés está en otro sitio. Seis características temporales por sí solas, sin mirar un byte de contenido ni un escalar de tamaño, alcanzan 0.9914 en el día de la botnet. Eso confirma que la periodicidad es una firma real y suficiente, y que el trabajo la estaba detectando por otra vía. Y como no leen el contenido, son invariantes ante el camuflaje de bytes de la sección 5.3, igual que la ráfaga.

Qué característica hace el trabajo, y una corrección

La ablación individual, en la tabla 5.7, mide dos cosas por cada característica: cuánto se pierde al quitarla del conjunto y cuánto consigue ella sola.

Tabla 5.7: Para cada característica, macro-F1 del conjunto sin ella, caída resultante, y macro-F1 que consigue por sí sola. Las seis juntas dan 0.9914.

Característica	Sin ella	Caída	Ella sola
dt_prev	0.9896	+0.0018	0.9669
dt_next	0.9900	+0.0014	0.9641
media_local	0.9789	+0.0125	0.9638
cv_local	0.9835	+0.0079	0.9355
regularidad	0.9949	−0.0035	0.9062
frac_cerca_mediana	0.9887	+0.0026	0.9336

El resultado corrigió una afirmación previa de este trabajo. Se había escrito que el coeficiente de variación local era la característica del *beaconing*, porque mide precisamente la regularidad de los intervalos y esa es la intuición evidente. La ablación lo desmiente por

partida doble: la característica cuya ausencia más cuesta es la media local, con una caída de 0.0125 frente a 0.0079, y la que mejor funciona en solitario es el intervalo anterior en crudo, con 0.9669.

Hay además una característica que estorba. Quitar regularidad mejora el resultado en 0.0035, y en solitario es la peor de las seis con 0.9062. Es redundante con el coeficiente de variación y añade ruido, así que debería eliminarse del conjunto.

La lectura de conjunto es que la señal está en los intervalos mismos y no en los estadísticos que se construyen sobre ellos. Un solo intervalo en crudo alcanza 0.9669, y las cinco características derivadas añaden en total dos puntos y medio. Es un recordatorio útil frente a la tentación de multiplicar características derivadas: cada una necesita medirse antes de incorporarla, porque alguna puede restar.

5.8. La fuga de identificadores de red

La tercera perturbación descubre un problema que las dos anteriores no podían ver, y que además sobrevive a la validación entre días.

Si a la vista de metadatos se le añaden las direcciones IP y el puerto de origen, algo que buena parte de la literatura hace porque están disponibles en el registro de flujo, el resultado entre días se vuelve perfecto: macro-F1 de 1.0000 en cinco de los seis pares con Random Forest y con XGBoost.

Parece una mejora y es un atajo, como cuantifica la figura 5.6. El mecanismo se ve al mirar el direccionamiento del laboratorio: de las 18 direcciones atacantes que aparecen en los seis días, 15 empiezan por 18., y las 12 víctimas están todas en 172.31.69.x. Un modelo que aprenda que un origen 18.x implica ataque transfiere perfectamente entre cualquier par de días del conjunto, sin haber aprendido nada sobre ataques.

La parte que importa metodológicamente es que la validación entre días no detecta el atajo. Al contrario, lo premia. Solo se rompe al cambiar de conjunto de datos, donde el direccionamiento es distinto, y allí la vista de metadatos cae al azar. Es el mismo problema que Engelen y sus coautores documentan en CIC-IDS2017 [7], aquí medido sobre CSE-CIC-IDS2018 y con una consecuencia añadida: ninguna de las validaciones habituales dentro de un mismo conjunto de datos lo detecta.

Un detalle refuerza el riesgo. Dos direcciones se reutilizan entre días con papeles distintos: 18.219.211.138 es atacante de DoS-GoldenEye el 15-02 y canal de mando de la botnet el 02-03, y 18.218.115.60 ejecuta los ataques web el 22-02 y es uno de los diez orígenes del DDoS el 20-02. Un modelo que memorice direcciones acertará en ambos días por el motivo equivocado.

Por eso este trabajo excluye las direcciones y los puertos de todas las vistas del capítulo 4, y por eso ese mismo capítulo incluye la dirección IP únicamente como control positivo declarado, nunca como característica.

5.9. Generalización entre conjuntos de datos

La cuarta perturbación es la más exigente: entrenar en CSE-CIC-IDS2018 y evaluar en CIC-IDS2017, sobre el ataque análogo. Cambia la red, el direccionamiento, el tráfico de

fondo y el año. Por indicación de la dirección del trabajo esta línea quedó como validación complementaria, pero el capítulo la necesita, porque es la única que rompe la fuga de la sección anterior.

Entrenando en 2018 y evaluando en 2017, el byte-CNN alcanza 0.9914 en el régimen web, 0.9990 en el cifrado y 0.9656 en el volumétrico, mientras la vista de metadatos cae a 0.5287, 0.5139 y 0.4291, es decir, al azar. La lectura inmediata sería que el contenido generaliza y los metadatos no.

Después de la sección 5.2 esa lectura ya no basta, y hay que preguntar por qué transfiere cada caso:

- El régimen web transfiere por contenido intrínseco. Las cadenas `union select` y `<script>` aparecen en ambos conjuntos porque son el ataque. Es transferencia robusta.
- El cifrado transfiere porque los atacantes de ambos años usan la misma biblioteca y emiten el mismo banner paramiko. Es un *fingerprint* compartido, y por tanto evadible.
- El volumétrico, igual, por los *User-Agent* y las cabeceras de Hulk y GoldenEye, comunes a ambos años.

Dos de los tres transfieren por el motivo equivocado. Un *fingerprint* compartido entre conjuntos de datos sigue siendo un *fingerprint*: que generalice no lo hace robusto, solo lo hace consistentemente frágil.

Una asimetría honesta

La dirección contraria no funciona. Entrenando en 2017 y evaluando en 2018, el byte-CNN colapsa: 0.5000 en el régimen web, donde predice una sola clase, 0.6051 en el cifrado y 0.6710 en el volumétrico. El motivo es la diversidad, porque los ataques de 2018 son más numerosos y más variados, de modo que un modelo entrenado allí ha visto más y transfiere, mientras que al revés no.

Es un argumento a favor de entrenar con la mayor variedad disponible, y también un recordatorio de que la generalización cruzada no es simétrica: reportar solo la dirección que funciona sería una forma elegante de mentir.

5.10. ¿Mejoran las arquitecturas del estado del arte?

Queda una pregunta que atraviesa todo el trabajo: si los resultados no son mejores, cabe preguntarse si es porque los modelos empleados se quedan cortos. Para responderla se implementaron dos arquitecturas del estado del arte y se compararon con las propias a presupuesto de parámetros comparable.

El transformer nunca gana

Sobre la secuencia de bytes se comparan byte-CNN, byte-LSTM y byte-Transformer. El resultado depende de la cantidad de datos y no favorece al modelo más moderno. Con pocos datos, en el día web con 203 flujos por clase, el *transformer* es peor: 0.9778 frente

5.10. ¿Mejoran las arquitecturas del estado del arte?

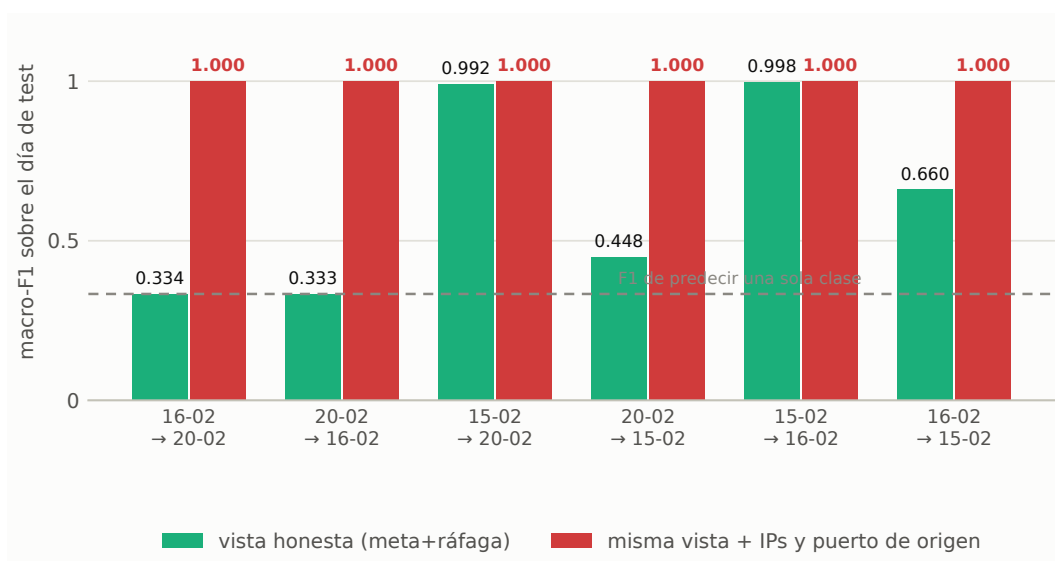


Figura 5.6: Macro-F1 entre días al añadir las direcciones y el puerto de origen a la vista de metadatos. La validación entre días no detecta el atajo: lo premia.

a 0.9975 del byte-CNN. Con datos suficientes, en el día de la botnet con 4.000 por clase, empata en 1.0000. Y siempre cuesta entre 36 y 49 veces más tiempo, con 3.191 segundos frente a 65 en el día de la botnet.

La figura 5.7 contrapone rendimiento y coste. El byte-LSTM, por su parte, se hunde con pocos datos hasta 0.8296, lo que es coherente con lo que mostró la interpretabilidad: la firma vive en los primeros bytes, y una convolución con agrupación máxima la localiza mejor que una recurrencia que ha de recorrer toda la secuencia.

La atención cruzada empata usando el doble de parámetros

En la fusión de las dos ramas, la atención cruzada iguala a la fusión tardía, con 0.9828 frente a 0.9828 en el día web y 0.9999 frente a 1.0000 en el de la botnet, empleando 72.066 parámetros frente a 31.042, es decir, 2.3 veces más. No hay ganancia que justifique el coste.

Y no es por falta de ajuste

Cabría objetar que los modelos propios están mejor ajustados que los tomados de la literatura. Se comprobó con una búsqueda aleatoria de hiperparámetros de 30 configuraciones, validada únicamente dentro del día de entrenamiento para no contaminar la evaluación. La carencia era real pero pequeña: el mejor ajuste mejora el MLP en +0.0275 en el día web y en +0.0063 en el volumétrico, y a XGBoost en +0.0122 y +0.0005. Nada de eso cambia ninguna conclusión.

Esa comprobación tiene un alcance acotado. La búsqueda cubre los dos modelos tabulares, el perceptrón multicapa y XGBoost, sobre los que se apoya la mayor parte de las mediciones del trabajo. Las cinco redes profundas conservan hiperparámetros fijos: el argumento vale para el ajuste de los modelos tabulares y no es una búsqueda exhaustiva sobre las arquitecturas profundas. Lo que sí queda igualado entre ellas, que es lo que la comparación del apartado mide, es el presupuesto de parámetros.

Qué se sigue de esto

Ninguna de las arquitecturas avanzadas mejora los resultados de este trabajo sobre estos datos, y dos de ellas cuestan entre 2 y 49 veces más. No es un argumento contra el estado del arte, cuyos modelos están pensados para escalas y escenarios mayores, y esta comparación no los refuta en general.

Es, en cambio, la confirmación de lo que el capítulo 4 había medido por otra vía. Si dentro del día la evaluación está saturada, no queda margen que una arquitectura mejor pueda recoger. El cuello de botella de este problema no está en la capacidad del modelo, sino en la calidad de la etiqueta, en los atajos que el conjunto de datos permite y en cómo se mide la transferencia. Que es donde este trabajo ha intervenido.

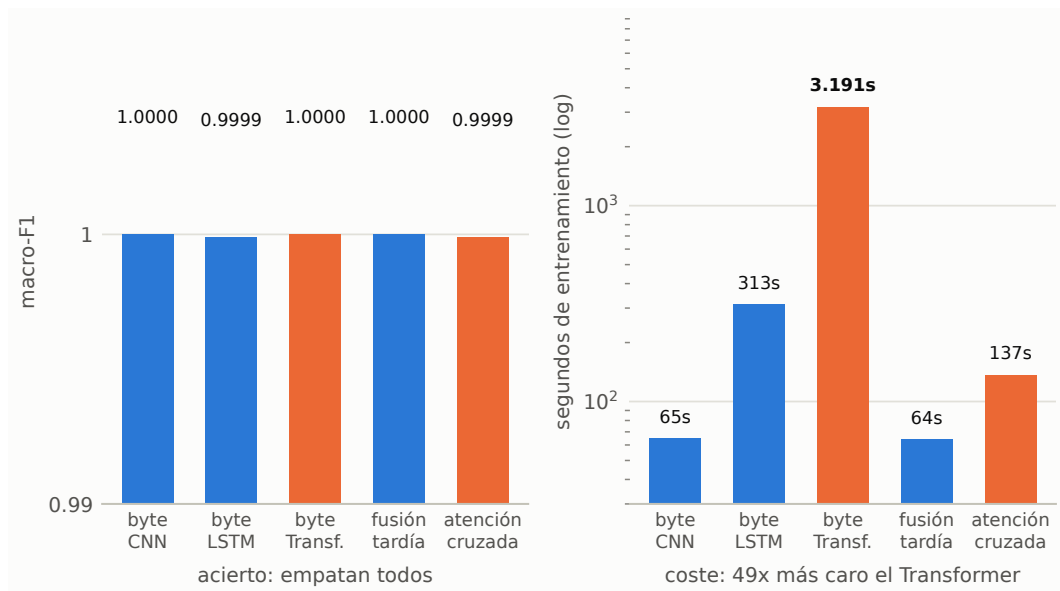


Figura 5.7: Macro-F1 y coste computacional a presupuesto de parámetros comparable. El *transformer* nunca gana y siempre cuesta más.

Conclusiones

6.1. Cumplimiento de los objetivos

Los seis objetivos de la sección 1.3 se cierran a continuación con la evidencia que los respalda. Dos se cumplieron de forma distinta a la prevista y se indica cómo.

O1. Pipeline reproducible. Cumplido. Siete días procesados desde la captura en bruto hasta el conjunto etiquetado, con la conexión TCP como unidad y cinco representaciones por conexión (sección 3.1). El almacenamiento es autocontenido, lo que permitió derivar dos vistas nuevas, la ráfaga y el inter-arribo, sin volver a procesar unas capturas que ya se habían borrado por falta de espacio en disco.

O2. Ground-truth verificada. Cumplido, y con hallazgo. La etiqueta de los siete días se reconstruyó sobre las capturas y se contrastó con la documentación oficial, que resultó incorrecta en tres de ellos: la dirección del atacante SSH del 14-02, el puerto del ataque lento del 16-02 y la continuidad de la actividad de la botnet del 02-03 (sección 3.5).

O3. Comparación con protocolo único. Cumplido, con el resultado opuesto al esperado. Las tres representaciones se compararon con la misma métrica, el mismo balanceo y el mismo tratamiento del benigno sobre los seis días. El objetivo pedía establecer si la comparación discrimina, y la respuesta es que no lo hace: ninguna vista baja de 0.9186 y el análisis de contenido no baja de 0.9734 (sección 4.3, tabla 4.3). El objetivo se cumple; la hipótesis que lo motivaba, no.

O4. Por qué acierta cada representación. Cumplido. La saliencia localiza el 31.1 % de la importancia en los primeros 48 bytes, que es la zona del *banner* en claro (sección 5.2), y la prueba de evasión lo confirma hundiendo el *recall* del contenido de 1.0000 a 0.0025 en el día de la botnet, con la excepción del régimen en claro (sección 5.3, tabla 5.1).

O5. Generalización. Cumplido, y con corrección de rumbo. Se midió entre los seis pares de días volumétricos y entre los dos conjuntos de datos. El componente que falla está identificado, y no es la representación sino el umbral: el AUC no baja de 0.9724 mientras

el macro-F1 con umbral fijo da 0.3333 (sección 5.6, tabla 5.4). Se aporta además la receta correspondiente, que son 25 etiquetas del dominio nuevo.

O6. Contraste con el estado del arte. Cumplido. Se implementaron un *transformer* sobre bytes y una red de atención cruzada, y se compararon a presupuesto de parámetros comparable. Ninguna mejora los resultados (sección 5.10). El límite no está en la capacidad del modelo.

6.2. Conclusiones principales

La evaluación dentro del día está saturada y no discrimina. Es la conclusión que reordenó el trabajo. Sobre estos conjuntos de datos, cualquiera de las representaciones detecta cualquiera de los ataques, así que un macro-F1 alto obtenido así no mide capacidad de detección. La causa que primero se propuso, la memorización del host atacante, se contrastó y resultó falsa: lo que las vistas aprenden es la herramienta con que se generó el ataque.

La detección robusta sí exige combinar contenido y conducta, pero quien lo demuestra no es la comparación de aciertos. Lo demuestran la evasión, que hunde el contenido salvo donde la firma es genuina, la transferencia entre dominios y los ataques que no dejan contenido que analizar. La tesis inicial del trabajo sobrevive; la evidencia que la sostiene es otra.

El éxito del análisis de contenido suele ser una huella de la herramienta, no del ataque. Un atacante lo anula cambiando de biblioteca, sin modificar su técnica. La excepción es el régimen en claro, donde los bytes que el modelo usa son el ataque, y por eso resiste.

Hay ataques estructuralmente invisibles al análisis de contenido. No son ataques difíciles, son invisibles: el 94.4 % de las conexiones del reconocimiento de la Infiltration no lleva un solo byte de aplicación. Ninguna mejora del modelo lo arregla, porque no hay datos que mejorar. Es el argumento más fuerte a favor de combinar vistas y el único que no depende de comparar métricas.

Entre dominios lo que se rompe primero es la calibración. Y se arregla con 25 etiquetas, sin reentrenar. Es la conclusión con aplicación práctica más directa de toda la memoria.

Sin ataques etiquetados, el rendimiento cae entre veinte y cincuenta puntos. Todas las cifras supervisadas de esta memoria suponen disponer de ejemplos etiquetados del mismo tipo de ataque. Al retirar esa suposición y entrenar solo con tráfico benigno, el área bajo la curva ROC baja a 0.6946 en el régimen cifrado y a 0.5402 en el volumétrico (sección 5.5). Esa es la distancia real entre lo que se mide sobre un conjunto etiquetado y lo que un detector encontraría ante una amenaza nueva.

El enfoque híbrido aporta por dónde se une, no por disponer de las dos fuentes. Concatenar las entradas crudas da un resultado peor que la mejor de las dos ramas por separado, porque las 257 dimensiones del contenido diluyen los cuatro escalares de flujo. La fusión tardía lo evita (sección 5.4).

El cuello de botella está en el dato y en el protocolo, no en el modelo. Ninguna arquitectura avanzada mejoró nada, y ahora se sabe por qué: no quedaba margen que recoger.

6.3. Lecciones metodológicas

Esta sección recoge los errores que el trabajo cometió y corrigió. Se incluye porque cada uno produjo un resultado plausible antes de ser detectado, y esa es la clase de fallo que una memoria no debería ocultar.

El tráfico benigno contaminado, dos veces. En el día del SSH, 1.589 de los 2.022 flujos etiquetados como benignos eran del propio atacante; en el del DoS, el 65.1 % del benigno HTTP. Se estaba comparando el ataque consigo mismo. El error produjo un techo de 0.78 que durante semanas se interpretó como una propiedad del problema. Un número bajo no levanta sospechas del mismo modo que un 1.0000, y ahí está la trampa.

Una ley inducida de dos puntos. El comportamiento de la generalización entre días se enunció con dos días, resultó falso al añadir el tercero y volvió a cambiar al medirlo con AUC en lugar de con F1. Tres versiones, dos de ellas equivocadas. La lección tiene dos partes: no inducir reglas con dos observaciones, y no juzgar la transferencia con una métrica que dependa de un umbral fijo.

Métricas que no eran lo que decían. Las cifras del caso difícil se citaron durante semanas como macro-F1 cuando eran exactitud en validación cruzada, en un capítulo que dedica una sección a explicar por qué la exactitud engaña. Y una figura comparaba dos conjuntos de características distintos bajo la misma etiqueta, lo que exageraba un efecto real. Antes de citar una cifra hay que comprobar qué métrica es y qué contiene.

Una intuición sobre las características que la ablación desmintió. Se había escrito que el coeficiente de variación local era la característica clave del *beaconing*, porque mide la regularidad de los intervalos y esa es la intuición evidente. La ablación mostró que la característica cuya ausencia más cuesta es la media local, que la mejor en solitario es el intervalo anterior en crudo, y que una de las seis, regularidad, empeora el conjunto (sección 5.7). Medir la aportación de cada característica antes de darla por buena, porque alguna puede restar.

Una hipótesis propia, contrastada y refutada. La explicación que se dio a la saturación, la memorización del host atacante, era razonable y era falsa. Se diseñó el experimento que podía tumbarla, se verificó con un control positivo que el experimento era sensible, y se tumbó (sección 4.4). Es el episodio del que más se aprendió.

Una consecuencia práctica. Estas correcciones motivaron una herramienta de verificación que contrasta cada cifra de la memoria contra la fuente de resultados, incluida la procedencia de las tablas. No sustituye a leer, pero impide que un número se quede desfasado en silencio.

6.4. Limitaciones

- Un solo conjunto de datos como eje. CIC-IDS2017 se usa como validación complementaria, no como segundo eje.
- Un único host atacante en cinco de los seis días. Es lo que impide contrastar la hipótesis del atajo por identidad de host más allá del día del DDoS. En los otros cinco queda sin contrastar.
- Las características de ráfaga cuentan conexiones por origen. Aunque el experimento de exclusión de atacantes muestra que generalizan a un host no visto, la reserva merece declararse.
- La evasión probada es de una sola clase, el camuflaje de los primeros bytes. Un atacante que espacie deliberadamente sus conexiones atacaría la vista conductual con el mismo esfuerzo, y ese caso no se ha medido. Ser invariante no equivale a ser robusto.
- La detección sin etiquetas de ataque se ensaya solo con dos métodos clásicos y sobre tres días. Es suficiente para acotar la distancia respecto al caso supervisado, pero no para concluir nada sobre el estado del arte en detección de anomalías.
- Se contrasta la arquitectura, no el preentrenamiento. La comparación con el *transformer* no incluye el preentrenamiento masivo, que es buena parte de lo que aporta ET-BERT.
- No hay día homólogo de botnet, con lo que el resultado del día 02-03 no está validado contra el test cruzado que sí se aplicó al DoS.
- Los grafos y los modelos de espacio de estados quedan fuera de alcance, y los primeros son justamente los que capturarían el caso que este trabajo identifica como su límite.

6.5. Trabajo futuro

Se ordenan por retorno esperado, justificando cada línea con un resultado de esta memoria y no con su interés general.

Protocolo que rompa el atajo en todos los días. El experimento de exclusión de atacantes solo pudo aplicarse al día con diez orígenes. Capturas con varios atacantes por escenario permitirían generalizarlo, y es la continuación directa de la sección 4.4.

Evasión sobre la vista conductual. Un atacante que ralentice su fuerza bruta es el equivalente conductual de camuflar el *banner*. Hasta medirlo, la robustez de la ráfaga está afirmada solo frente a una clase de evasión.

Redes sobre grafos para el movimiento lateral. Lo pide el resultado de la sección 4.6: el reconocimiento de la Infiltration es un patrón topológico, un origen contra 612 destinos y sin carga útil, que ninguna de las vistas de este trabajo puede representar.

Recalibración continua en despliegue. La sección 5.6 muestra que 25 etiquetas bastan para reajustar el umbral en un dominio nuevo. Convertirlo en un procedimiento periódico es una línea de ingeniería con resultado medible.

Validación entre conjuntos de datos como norma. No como complemento. Es la única de las validaciones aplicadas que rompe la fuga de identificadores (sección 5.8), y por tanto la única que garantiza que lo medido sea detección y no memorización del laboratorio.

Declaración del uso de IA Generativa

Herramienta/s utilizada/s:

Claude (Anthropic), utilizado a través de Claude Code sobre el repositorio del proyecto, desde las fases iniciales hasta la redacción de esta memoria (marzo a agosto de 2026).

Uso realizado (marcar lo que corresponda):

- ☒ Generación de texto
- ☒ Reformulación
- ☐ Traducción / corrección
- ☒ Sugerencia de estructura
- ☐ Apoyo metodológico
- ☒ Ayuda para codificar / programar
- ☒ Otros: generación del material gráfico

Descripción del uso realizado:

La herramienta se utilizó a lo largo de todo el proyecto en tres tareas. La primera es la escritura y depuración de los programas: los de extracción y vectorización del tráfico, los de evaluación y las implementaciones de las arquitecturas de aprendizaje profundo, tanto las propias (ByteCNN, ByteLSTM y el híbrido de dos ramas) como las tomadas del estado del arte para contrastarlas (ByteTransformer y CrossAttentionNet). La segunda es la generación del material gráfico: las figuras y las tablas de esta memoria se producen con programas escritos con esa asistencia. La tercera es la redacción, estructuración y reformulación de partes de la memoria y de la documentación técnica del repositorio.

Dirección y decisiones. La dirección del trabajo fue en todo momento del autor: el plan-

teamamiento del problema y la tesis central, la elección de los días del conjunto de datos a procesar, el orden de prioridades, el alcance de cada experimento y las decisiones sobre qué líneas abandonar o profundizar. Varias correcciones de rumbo, incluida la detección de resultados mal interpretados y la petición de contrastar hipótesis que resultaron falsas, partieron del autor.

Trazabilidad, y sus límites. Las contribuciones asistidas están identificadas en el historial de control de versiones mediante la marca Co-Authored-By. Esa marca se empezó a aplicar de forma sistemática con el proyecto ya avanzado, de modo que constituye una cota inferior y no un registro completo: hubo asistencia también en aportaciones anteriores que no la llevan. Esta declaración prevalece sobre el recuento del historial.

Compromiso de autoría:

Confirmando que todo el contenido generado ha sido revisado, reelaborado y validado personalmente.

Resultados secundarios

Este anexo recoge mediciones que sostienen los capítulos 4 y 5 pero que no hacen falta para seguir su argumento. Se separan aquí por eso, y no porque sean menos fiables: proceden de las mismas ejecuciones y del mismo módulo de resultados que las del cuerpo.

B.1. El *recall* detrás del macro-F1

La sección 4.1 declara que se reportan dos métricas, el macro-F1 y el *recall* de la clase de ataque, y explica por qué la segunda hace falta: un macro-F1 mediocre puede corresponder tanto a un clasificador que se equivoca como a uno que no detecta nada, y solo el *recall* distingue los dos casos. El cuerpo de la memoria recoge el macro-F1 en la tabla 4.3; la tabla B.1 completa el par.

La lectura confirma la saturación por la segunda vía. Ninguna vista deja escapar más del 3 % de los ataques en ningún día, de modo que el diagnóstico del capítulo 4 no depende de haber elegido una métrica concreta.

Tabla B.1: *Recall* de la clase de ataque en la evaluación intra-día. Complementa la tabla 4.3, que recoge el macro-F1 de las mismas mediciones.

Día	Payload	Metadatos	Conducta
14-02 cifrado	0.9997	0.9997	0.9997
22-02 en claro	0.9867	0.9887	0.9714
16-02 volumétrico	1.0000	0.9981	0.9998
15-02 volumétrico	0.9997	0.9876	0.9963
20-02 vol. diluido	1.0000	1.0000	0.9984
02-03 periodicidad	0.9987	0.9974	0.9988

B.2. Área bajo la curva con intervalos

La tabla B.2 recoge el AUC-ROC con intervalo de confianza al 95 % para los dos regímenes en que se midió con el conjunto benigno ya corregido. Es la métrica que la sección 5.6

necesita, porque separa dos preguntas que el macro-F1 confunde: si el modelo ordena bien los flujos y si el umbral heredado corta donde debe.

El día volumétrico no aparece porque su medición es anterior al reetiquetado, y las cifras de aquel escenario no son comparables con las de esta memoria por el motivo que expone la sección 3.6. Reportarlas junto a las demás sería mezclar dos conjuntos de datos distintos bajo la misma etiqueta, que es uno de los errores que este trabajo documenta haber cometido.

Tabla B.2: Área bajo la curva ROC con intervalo de confianza al 95 % de 2.000 remuestreos. Es la medida independiente del umbral que la sección 5.6 emplea para separar el fallo de representación del fallo de umbral.

Régimen	Vista	AUC-ROC (IC 95 %)
Cifrado (14-02, SSH)	payload byte-CNN	1.0000 [1.0000, 1.0000]
	payload histograma	0.9958 [0.9902, 1.0000]
	conducta meta + ráfaga	1.0000 [1.0000, 1.0000]
En claro (22-02, Web)	payload byte-CNN	1.0000 [1.0000, 1.0000]
	payload histograma	0.9949 [0.9843, 1.0000]
	conducta meta + ráfaga	0.9998 [0.9994, 1.0000]

B.3. Detección no supervisada en términos operativos

La tabla 5.3 del cuerpo reporta el área bajo la curva, que resume la calidad del orden pero no dice cuántos ataques se detectarían en la práctica. La tabla B.3 responde a esa pregunta: qué fracción del ataque se captura si se acepta un 5 % de falsos positivos, que es un presupuesto de alarmas realista para un analista.

La lectura es más dura que la del área bajo la curva y conviene retenerla. En el régimen cifrado el análisis de contenido no captura ni un solo ataque con ese presupuesto, pese a un AUC de 0.6946, mientras la conducta los captura todos. En el volumétrico ninguno de los tres sirve. Un AUC intermedio puede corresponder, por tanto, a un detector inservible en el punto de operación que importa, y esa es una advertencia sobre cómo se leen las cifras de detección de anomalías en la literatura.

Tabla B.3: Fracción de ataques detectados aceptando un 5 % de falsos positivos, entrenando únicamente con tráfico benigno. Es la lectura operativa de la tabla 5.3, que reporta el área bajo la curva.

Régimen	Payload (IF)	Payload (AE)	Conducta (IF)
Cifrado (14-02, SSH)	0.0000	0.0000	1.0000
En claro (22-02, Web)	0.2808	0.0739	0.5714
Volumétrico (16-02, DoS)	0.0000	0.0000	0.0170

B.4. Búsqueda de hiperparámetros

La sección 5.10 resume esta búsqueda en dos frases porque su conclusión es que no cambia nada. La tabla B.4 recoge los cuatro casos completos, incluido el coste, que también informa: la búsqueda sobre el perceptrón multicapa en el día volumétrico tarda 2.624 segundos frente a los 38 de XGBoost sobre el mismo problema, es decir, casi setenta veces más para una ganancia de 0.0063 frente a 0.0005.

La búsqueda cubre los dos modelos tabulares y se valida únicamente dentro del día de entrenamiento. El día de prueba no participa en la elección, porque ajustar mirando el conjunto de prueba es precisamente el defecto que esta memoria denuncia en otros sitios.

Tabla B.4: Búsqueda aleatoria de 30 configuraciones, validada únicamente dentro del día de entrenamiento. Macro-F1 con los valores por defecto, con los mejores encontrados, y coste de la búsqueda.

Caso	Modelo	Por defecto	Mejor	Segundos
22-02 Web, metadatos	MLP	0.9107	0.9383	37
	XGBoost	0.9556	0.9679	46
15-02 DoS, metadatos	MLP	0.9556	0.9619	2.624
	XGBoost	0.9814	0.9819	38

Bibliografía

- [1] Mayank Swarnkar and Shyam Singh Rajput, editors. *Artificial Intelligence for Intrusion Detection Systems*. Chapman and Hall/CRC, 1 edition, 2023. Referencia general de encuadre del campo. Ver páginas 1, 6.
- [2] Ke Wang and Salvatore J. Stolfo. Anomalous payload-based network intrusion detection. In *Recent Advances in Intrusion Detection (RAID)*, pages 203–222, 2004. PAYL: distribucion de bytes de la carga util. Ver páginas 3, 13, and 17.
- [3] Ke Wang, Janak J. Parekh, and Salvatore J. Stolfo. Anagram: A content anomaly detector resistant to mimicry attack. In *Recent Advances in Intrusion Detection (RAID)*, pages 226–248, 2006. Ver páginas 3, 13, and 18.
- [4] Mohammad Lotfollahi, Mahdi Jafari Siavoshani, Ramin Shirali Hossein Zade, and Mohammad-sadegh Saberian. Deep packet: A novel approach for encrypted traffic classification using deep learning. *Soft Computing*, 24:1999–2012, 2020. Referencia del byte-CNN empleado en este trabajo. Ver páginas 3, 13.
- [5] John C. Platt. Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In *Advances in Large Margin Classifiers*, pages 61–74. MIT Press, 1999. Metodo de recalibracion empleado en la seccion 5.5. Ver páginas 3, 13, and 46.
- [6] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 1321–1330, 2017. Origen del Expected Calibration Error (ECE). Ver páginas 3, 13.
- [7] Gints Engelen, Vera Rimmer, and Wouter Joosen. Troubleshooting an intrusion detection dataset: the CICIDS2017 case study. In *IEEE Security and Privacy Workshops (SPW), Workshop on Traffic Measurements for Cybersecurity (WTMC)*, 2021. Origen del *shortcut learning* y de las etiquetas *Attempted*. Código: <https://github.com/GintsEngelen/WTMC2021-Code>. Ver páginas 3, 12, 13, 21, 33, 37, and 49.
- [8] Canadian Institute for Cybersecurity and Communications Security Establishment. CSE-CIC-IDS2018 on AWS. <https://www.unb.ca/cic/datasets/ids-2018.html>, 2018. Conjunto de datos principal de este trabajo. Ver páginas 4, 11.
- [9] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016. Ver páginas 7, 40.
- [10] Sercan Ö. Arik and Tomas Pfister. TabNet: Attentive interpretable tabular learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 6679–6687, 2021. Ver página 8.
- [11] Demóstenes Zegarra Rodríguez, Ogobuchi Daniel Okey, Siti Sarah Maidin, Ekikere Umoren Udo, and João Henrique Kleinschmidt. Attentive transformer deep learning algorithm for intrusion detection on IoT systems using automatic explainable feature selection. *PLOS ONE*, 18(10):e0286652, 2023. TabNet-IDS: 97.03 % en CIC-IDS2017 y 95.58 % en CSE-CIC-IDS2018. Ver página 8.

-
- [12] Wai Weng Lo, Siamak Layeghy, Mohanad Sarhan, Marcus Gallagher, and Marius Portmann. E-GraphSAGE: A graph neural network based intrusion detection system for IoT. In *IEEE/IFIP Network Operations and Management Symposium (NOMS)*, pages 1–9, 2022. Ver página 8.
- [13] Wei Wang, Ming Zhu, Xuewen Zeng, Xiaozhou Ye, and Yiqiang Sheng. Malware traffic classification using convolutional neural network for representation learning. In *International Conference on Information Networking (ICOIN)*, pages 712–717, 2017. Origen de la linea que rasteriza el payload como imagen 2D. Ver página 9.
- [14] K. Harinath and G. Kishor Kumar. An efficient approach for encrypted traffic classification and intrusion detection using packet transformer encoder and CNN. *International Journal of Computational and Experimental Science and Engineering*, 11(3), 2025. Ver página 9.
- [15] Gayan K. Kulatilleke, Siamak Layeghy, Mahsa Baktashmotlagh, and Marius Portmann. MambaNetBurst: Direct byte-level network traffic classification without tokenization or pretraining. Preprint arXiv:2605.11034, 2026. <https://arxiv.org/abs/2605.11034>. Ver página 9.
- [16] L. Xiong, L. Wan, and J. Ren. Two-branch intrusion detection method based on fusion of deep semantic and statistical features. *Computers, Materials & Continua*, 87(3), 2026. CPS-IDS: fusion de payload y estadísticas por atencion cruzada. Ver página 10.
- [17] Salam Allawi Hussein and Sándor R. Répás. A hybrid intrusion detection framework using deep autoencoder and machine learning models. *AI*, 7(2):39, 2026. AE-GMM con XGBoost y regresion logistica. Ver página 10.
- [18] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. Toward generating a new intrusion detection dataset and intrusion traffic characterization. In *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, pages 108–116, 2018. Presenta CIC-IDS2017 y la herramienta CICFlowMeter. Ver página 11.
- [19] Nour Moustafa and Jill Slay. UNSW-NB15: A comprehensive data set for network intrusion detection systems. In *Military Communications and Information Systems Conference (MilCIS)*, pages 1–6, 2015. Ver página 11.
- [20] Brad Duncan. Malware-traffic-analysis.net. <https://www.malware-traffic-analysis.net/>. Capturas de infecciones reales; no usado en este trabajo. Ver página 11.
- [21] The Zeek Project. The Zeek network security monitor. <https://zeek.org/>. Extraccion de carga util y metadatos de flujo. Ver página 15.
- [22] Wenke Lee and Salvatore J. Stolfo. A framework for constructing features and models for intrusion detection systems. *ACM Transactions on Information and System Security*, 3(4):227–261, 2000. Origen de la taxonomia de características intrínsecas, de contenido y de trafico. Ver página 16.
- [23] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011. Ver página 40.
- [24] Adam Paszke, Sam Gross, Francisco Massa, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*, pages 8024–8035, 2019. Ver página 40.
- [25] Chaonan Xin and Keqing Xu. Cross-dataset transformer-IDS with calibration and AUC optimization. *Journal of Cyber Security*, 7(1):483–503, 2025. Corroboracion independiente del HALLAZGO 14: mismo fenomeno de descalibracion entre dominios, corregido con escalado de Platt. Ver página 46.